**TRƯỜNG ĐẠI HỌC NGOẠI NGỮ - TIN HỌC TP. HỒ CHÍ MINH**
**KHOA CÔNG NGHỆ THÔNG TIN**

---

**HỌC PHẦN: HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU**

**(Oracle Database 23ai)**

## ĐỀ TÀI ĐỒ ÁN CUỐI KỲ:

# HỆ THỐNG THANH TOÁN VÍ ĐIỆN TỬ NỘI BỘ DOANH NGHIỆP

**(Ứng dụng PL/SQL, Functions, Triggers và Xử lý Ngoại lệ trên Oracle 23ai)**

**GIẢNG VIÊN HƯỚNG DẪN:** **ThS. Lê Thị Minh Nguyện**

**NHÓM THỰC HIỆN:** **Nhóm 12 (Đề tài số 08)**

**SINH VIÊN THỰC HIỆN:** **Nguyễn Quốc Huy – 24DH113268 (Nhóm trưởng)**

Phạm Thành Đức – 24DH113249

Nguyễn Thế Vinh – 24DH141906

**TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026**

# LỜI CẢM ƠN

Trong suốt thời gian học tập và nghiên cứu học phần Hệ quản trị Cơ sở dữ liệu tại Khoa Công nghệ Thông tin, Trường Đại học Ngoại ngữ – Tin học TP.HCM (HUFLIT), nhóm chúng em đã được tiếp cận với nền tảng kiến thức chuyên sâu và hiện đại về kiến trúc Oracle Database 23ai, ngôn ngữ lập trình thủ tục PL/SQL, các cơ chế bảo vệ toàn vẹn giao dịch, kiểm soát đồng thời và tối ưu hoá truy vấn.

Nhóm 12 xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Lê Thị Minh Nguyện, giảng viên phụ trách học phần, người đã tận tình giảng dạy, truyền đạt kinh nghiệm thực tiễn quý báu và định hướng chi tiết để nhóm hoàn thiện đề tài "Hệ thống thanh toán ví điện tử nội bộ doanh nghiệp" từ khâu thiết kế lược đồ cơ sở dữ liệu, cài đặt mã nguồn PL/SQL trên Docker/DBeaver đến xây dựng ứng dụng web minh hoạ hoàn chỉnh.

Mặc dù đã nỗ lực kiểm thử cẩn thận trên từng tình huống biên và tối ưu hoá hiệu năng truy vấn, báo cáo và sản phẩm khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ Cô để hoàn thiện hơn nữa kiến thức và kỹ năng nghề nghiệp.

# NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

# MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI ..... 1
1.1. Đặt vấn đề và bối cảnh doanh nghiệp ..... 1
1.2. Lý do chọn đề tài ..... 1
1.3. Mục tiêu đề tài ..... 2
1.3.1. Mục tiêu tổng quát ..... 2
1.3.2. Mục tiêu cụ thể ..... 2
1.4. Đối tượng và phạm vi nghiên cứu ..... 3
1.5. Danh sách công nghệ Oracle Database 23ai sử dụng và lý do lựa chọn ..... 3
1.6. Kế hoạch thực hiện và phân công ban đầu của Nhóm 12 ..... 4
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG ..... 5
2.1. Phân tích quy trình nghiệp vụ ví điện tử nội bộ ..... 5
2.2. Các tình huống biên và bài toán ngoại lệ đặc thù ..... 6
2.3. Mô hình thực thể kết hợp (ERD) và chuẩn hoá 3NF ..... 7
2.4. Đặc tả chi tiết 9 bảng dữ liệu quan hệ và ràng buộc toàn vẹn ..... 8
2.5. Thiết kế kiến trúc xử lý ngoại lệ đa tầng và chống tranh chấp đồng thời ..... 10
CHƯƠNG 3. CÀI ĐẶT HỆ THỐNG VÀ TRIỂN KHAI ..... 13
3.1. Triển khai Oracle Database 23ai trên Docker và cấu hình DBeaver ..... 13
3.2. Cài đặt Package quản lý ngoại lệ PKG_NGOAI_LE_VI ..... 14
3.3. Cài đặt các Stored Functions và khả năng gọi trong câu lệnh SQL ..... 15
3.4. Cài đặt Package nghiệp vụ cốt lõi PKG_QUAN_LY_VI ..... 16
3.5. Cài đặt Triggers bảo vệ số dư và Compound Trigger chống Mutating Table ..... 18
3.6. Xây dựng lớp API Backend và Giao diện Web minh hoạ ..... 19
CHƯƠNG 4. KIỂM THỬ VÀ TỐI ƯU HOÁ ..... 22
4.1. Môi trường và tập dữ liệu kiểm thử ..... 22

4.2. Bảng kịch bản kiểm thử và kết quả xử lý ngoại lệ .....22
4.3. Phân tích và tối ưu hoá Truy vấn 1 (Thống kê giao dịch 30 ngày) .....24
4.4. Phân tích và tối ưu hoá Truy vấn 2 (Đối soát Nợ/Có sổ cái 90 ngày).....24
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN .....27
5.1. Kết quả đạt được.....27
5.2. Hạn chế và hướng phát triển.....27
5.3. Phân công và đóng góp của từng thành viên Nhóm 12.....28
5.4. Định hướng chuẩn bị vấn đáp theo Ngân hàng câu hỏi học phần.....28
TÀI LIỆU THAM KHẢO .....30
PHỤ LỤC: HƯỚNG DẪN TRIỂN KHAI DOCKER, DBEAVER VÀ KỊCH BẢN DEMO .....31
Phụ lục A. Trình tự triển khai hệ thống trên Docker và DBeaver.....31
Phụ lục B. Kịch bản Demo trực tiếp khi bảo vệ đồ án (8–10 phút) .....32

# DANH MỤC TỪ VIẾT TẮT

*Bảng 0.1. Danh mục các từ viết tắt sử dụng trong báo cáo*

| Từ viết tắt | Thuật ngữ tiếng Anh / Đầy đủ | Ý nghĩa trong đề tài |
|---|---|---|
| ACID | Atomicity, Consistency, Isolation, Durability | 4 tính chất đảm bảo an toàn giao dịch CSDL |
| API | Application Programming Interface | Giao diện lập trình ứng dụng kết nối tầng Web và CSDL |
| CBO | Cost-Based Optimizer | Bộ tối ưu hoá truy vấn dựa trên chi phí của Oracle |
| CSDL | Cơ sở dữ liệu | Tập hợp dữ liệu có cấu trúc được quản lý bởi DBMS |
| DDL | Data Definition Language | Ngôn ngữ định nghĩa cấu trúc dữ liệu (CREATE, ALTER...) |
| DML | Data Manipulation Language | Ngôn ngữ thao tác dữ liệu (INSERT, UPDATE, DELETE) |
| ERD | Entity-Relationship Diagram | Sơ đồ thực thể kết hợp mô hình hoá dữ liệu |
| FK / PK | Foreign Key / Primary Key | Khoá ngoại / Khoá chính trong bảng quan hệ |
| JSON | JavaScript Object Notation | Định dạng dữ liệu trao đổi nhẹ giữa API và giao diện |
| OLTP | Online Transaction Processing | Hệ thống xử lý giao dịch trực tuyến tốc độ cao |
| PDB | Pluggable Database | CSDL có thể cắm/rút trong kiến trúc Multitenant Oracle 23ai |
| PL/SQL | Procedural Language/Structured Query Language | Ngôn ngữ lập trình thủ tục mở rộng SQL của Oracle |
| REST | Representational State Transfer | Kiến trúc thiết kế API thông qua giao thức HTTP |
| 3NF | Third Normal Form | Dạng chuẩn 3 loại bỏ phụ thuộc bắc cầu trong thiết kế CSDL |

# DANH MỤC BẢNG BIỂU

Bảng 0.1. Danh mục các từ viết tắt sử dụng trong báo cáo .....v

Bảng 1.1. Tổng hợp công nghệ Oracle 23ai và kiến trúc sử dụng trong đồ án .....3

Bảng 2.1. Danh mục 10 ngoại lệ trọng tâm và cơ chế xử lý trong Oracle 23ai .....6

Bảng 2.2. Đặc tả bảng PHONG_BAN và NHAN_VIEN .....8

Bảng 2.3. Đặc tả bảng VI_DIEN_TU, DOI_TAC và LOAI_GIAO_DICH .....9

Bảng 2.4. Đặc tả bảng GIAO_DICH, CHI_TIET_SO_CAI và Audit Logs .....10

Bảng 3.1. Danh sách các Endpoint REST API và ánh xạ tới đối tượng Oracle 23ai....19

Bảng 4.1. Tổng hợp 9 kịch bản kiểm thử chức năng, ngoại lệ và kết quả xác minh thực tế .....22

Bảng 4.2. Kết quả đo Execution Plan thực tế trên Oracle Database 23ai (sql/10_optimization_plans.sql) .....25

Bảng 5.1. Tổng hợp đóng góp chi tiết của các thành viên Nhóm 12 .....28

Bảng A.1. Kịch bản trình bày và demo trực tiếp hệ thống trong buổi bảo vệ.....32

# DANH MỤC HÌNH ẢNH

Hình 1.1. Kiến trúc tổng thể 3 tầng hệ thống ví điện tử nội bộ trên Oracle Database 23ai .....4

Hình 2.1. Sơ đồ thực thể kết hợp (ERD) 9 bảng chuẩn 3NF của hệ thống .....8

Hình 2.2. Lưu đồ luồng xử lý giao dịch và kiến trúc Exception Handling đa tầng .....12

Hình 3.1. Cơ chế Compound Trigger ngăn ngừa lỗi Mutating Table (ORA-04091)....19

Hình 3.2. Giao diện Dashboard thực tế kết nối Oracle Database 23ai Free (FREEPDB1).....21

Hình 3.3. Kiểm thử nhanh ngoại lệ (-20005) và Nhật ký lỗi trên giao diện Web.....21

Hình 4.1. Kết quả đối soát Explicit Cursor trên 10 ví (0 ví lệch) qua giao diện Web ..23

Hình 4.2. Biểu đồ đo thực tế mức giảm Optimizer Cost và Buffers trên Oracle Database 23ai.....26

# CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

## 1.1. Đặt vấn đề và bối cảnh doanh nghiệp

Trong quá trình chuyển đổi số nội bộ tại các doanh nghiệp quy mô vừa và lớn, nhu cầu chi tiêu tập trung và thanh toán không dùng tiền mặt của cán bộ nhân viên diễn ra hàng ngày với tần suất cao: thanh toán suất ăn tại căn tin, phí gửi xe thông minh, chi phí in ấn – văn phòng phẩm, tạm ứng công tác phí hoặc chuyển khoản chia sẻ chi phí giữa các đồng nghiệp. Nếu quản lý thủ công bằng tiền mặt hoặc chuyển khoản ngân hàng cá nhân rời rạc, phòng Kế toán – Tài chính sẽ gặp nhiều khó khăn trong công tác đối soát, kiểm soát hạn mức chi tiêu theo ngày của từng nhân sự cũng như dễ phát sinh sai sót khi hoàn tiền.

Giải pháp xây dựng Hệ thống thanh toán ví điện tử nội bộ doanh nghiệp (Corporate Internal E-Wallet) cho phép mỗi nhân viên và các đơn vị cung cấp dịch vụ nội bộ sở hữu một tài khoản ví điện tử tập trung. Tuy nhiên, bài toán tài chính – thanh toán mang tính nhạy cảm tuyệt đối về độ chính xác dữ liệu. Chỉ một lỗi nhỏ trong kiểm soát giao dịch đồng thời, mất kết nối mạng khiến người dùng bấm gửi lại yêu cầu (retry), hoặc lỗ hổng cho phép ví bị khoá thực hiện trừ tiền đều có thể làm sai lệch số dư hệ thống.

## 1.2. Lý do chọn đề tài

Xuất phát từ yêu cầu thực tế nêu trên và định hướng của Ngân hàng đề tài môn Hệ quản trị Cơ sở dữ liệu (Đề tài số 08 thuộc Nhóm 3: PL/SQL, Functions và Exception Handling), Nhóm 12 lựa chọn đề tài "Hệ thống thanh toán ví điện tử nội bộ doanh nghiệp" vì ba lý do cốt lõi sau:

- **Tính điển hình cao về xử lý ngoại lệ (Exception Handling):** Các nghiệp vụ nạp, rút, chuyển tiền và hoàn tiền đòi hỏi phải phát hiện và xử lý triệt để cả ngoại lệ định sẵn của Oracle (NO_DATA_FOUND, DUP_VAL_ON_INDEX, VALUE_ERROR, ORA-00054, ORA-30006) lẫn các ngoại lệ nghiệp vụ tự định nghĩa (số dư không đủ, giao dịch trùng lặp theo Idempotency Key, ví bị khoá, vượt hạn mức ngày).
- **Khả năng vận dụng toàn diện cấu trúc PL/SQL trên Oracle Database 23ai:** Đề tài cho phép phối hợp nhịp nhàng giữa Stored Functions (gồm kiểu BOOLEAN

chuẩn SQL mới trên Oracle 23ai), Stored Procedures, Packages, Explicit Cursors để đối soát hàng loạt, SAVEPOINT / ROLLBACK bảo toàn tính nguyên tử ACID, PRAGMA AUTONOMOUS_TRANSACTION để lưu vết lỗi ngay cả khi giao dịch chính bị huỷ bỏ, và Triggers cùng Compound Trigger để ngăn chặn lỗi Mutating Table (ORA-04091).

- **Tính ứng dụng thực tế và khả năng triển khai hiện đại:** Toàn bộ cơ sở dữ liệu được đóng gói bằng Docker container Oracle Database 23ai Free, chạy song song thuận tiện trên DBeaver và kết nối trực tiếp với ứng dụng Web 3 tầng thông qua Node.js Express và driver oracledb Thin Mode.

## 1.3. Mục tiêu đề tài

### 1.3.1. Mục tiêu tổng quát

Thiết kế và cài đặt hoàn chỉnh hệ thống thanh toán ví điện tử nội bộ doanh nghiệp trên nền tảng Oracle Database 23ai, đảm bảo tính toàn vẹn tài chính tuyệt đối (khớp 100% giữa số dư ví và sổ cái kế toán kép), xử lý an toàn mọi tình huống ngoại lệ và đồng thời, kèm theo ứng dụng web thao tác trực quan qua lớp API.

### 1.3.2. Mục tiêu cụ thể

- **Thiết kế CSDL chuẩn 3NF:** Xây dựng mô hình ER và lược đồ quan hệ gồm 9 bảng dữ liệu chuẩn hoá, thiết lập đầy đủ khoá chính, khoá ngoại, ràng buộc CHECK bảo vệ số dư không âm và chỉ mục tổng hợp phục vụ tối ưu truy vấn.
- **Cài đặt hệ thống xử lý ngoại lệ và PL/SQL chuẩn mực:** Xây dựng Package PKG_NGOAI_LE_VI ánh xạ 10 mã lỗi nghiệp vụ (-20001 đến -20010) qua PRAGMA EXCEPTION_INIT; 5 Stored Functions độc lập; Package PKG_QUAN_LY_VI xử lý Nạp, Rút, Chuyển tiền, Thanh toán, Hoàn tiền và Đối soát Explicit Cursor; cùng 3 Triggers (gồm 1 Compound Trigger).
- **Kiểm thử toàn diện và tối ưu hoá truy vấn:** Sinh dữ liệu lịch sử 15.000+ giao dịch (hơn 30.000 bút toán sổ cái), thực thi tối thiểu 8 kịch bản kiểm thử bao gồm cả race-condition song song và phân tích Execution Plan trước/sau tối ưu cho 2 truy vấn trọng tâm.

- **Xây dựng ứng dụng Web 3 tầng hoàn chỉnh:** Phát triển REST API bằng Node.js + Express + oracledb Thin Mode và giao diện Web cho phép thực hiện mọi nghiệp vụ, mô phỏng nhanh các ngoại lệ và đối soát trực tiếp với Oracle 23ai.

## 1.4. Đối tượng và phạm vi nghiên cứu

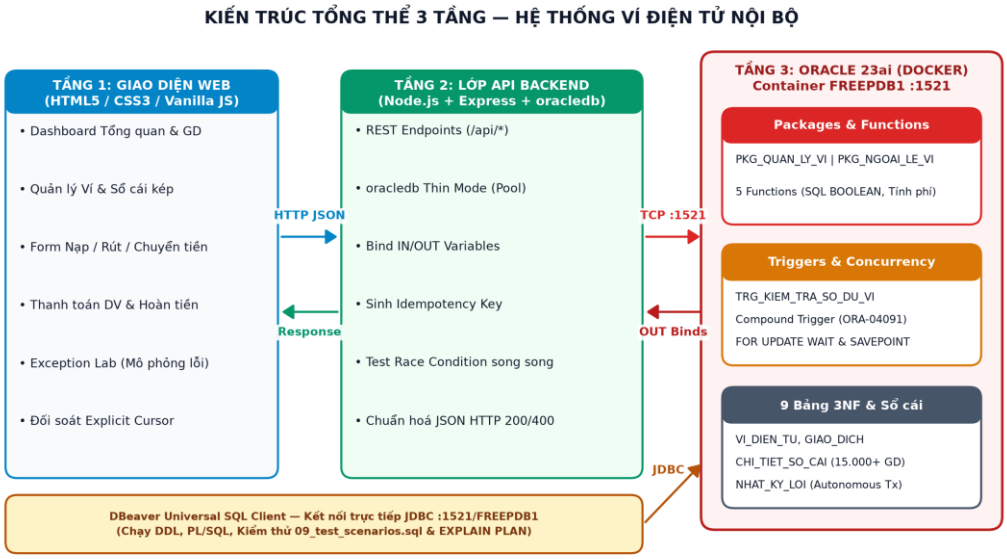
Đối tượng nghiên cứu của đồ án là các cơ chế lập trình phía máy chủ dữ liệu (Server-side Database Programming) trên hệ quản trị cơ sở dữ liệu Oracle Database 23ai, trọng tâm là kỹ thuật bắt và xử lý ngoại lệ trong giao dịch OLTP tài chính. Phạm vi bài toán giới hạn trong môi trường doanh nghiệp nội bộ với các thực thể: Phòng ban, Nhân viên, Ví điện tử (nhân viên và đối tác dịch vụ nội bộ), Loại giao dịch – biểu phí, Giao dịch, Sổ cái kế toán kép và các bảng Nhật ký kiểm toán (Audit Logs). Hệ thống tập trung vào luồng dữ liệu Giao diện Web ↔ API ↔ Oracle 23ai và bỏ qua các phân hệ phụ trợ ngoài phạm vi môn học như xác thực OAuth2 phức tạp hay tích hợp cổng thanh toán ngân hàng thật.

## 1.5. Danh sách công nghệ Oracle Database 23ai sử dụng và lý do lựa chọn

*Bảng 1.1. Tổng hợp công nghệ Oracle 23ai và kiến trúc sử dụng trong đồ án*

| Công nghệ / Tính năng | Vai trò trong hệ thống | Lý do lựa chọn |
|---|---|---|
| Oracle Database 23ai Free (Docker) | Hệ quản trị CSDL lõi (PDB FREEPDB1) | Phiên bản mới nhất theo yêu cầu môn học; hỗ trợ triển khai container hoá đồng nhất môi trường, bảng mã AL32UTF8 chuẩn tiếng Việt. |
| PRAGMA EXCEPTION_INIT (-20001..-20010) | Định nghĩa User-Defined Exceptions | Gắn tên ngoại lệ tường minh với mã lỗi Oracle tuỳ biến giúp mã PL/SQL dễ đọc, phân loại chính xác từng tình huống biên. |
| Predefined Exceptions (DUP_VAL_ON_INDEX, NO_DATA_FOUND, ORA-00054) | Bắt lỗi hệ thống Oracle tự động | Bảo vệ ở tầng ràng buộc vật lý ngay cả khi hai luồng đồng thời vượt qua bước kiểm tra SELECT ban đầu. |
| PRAGMA AUTONOMOUS_TRANSACTION | Ghi nhật ký lỗi độc lập | Cho phép COMMIT bản ghi vào NHAT_KY_LOI_GIAO_DICH trong khi giao dịch chuyển tiền chính bị ROLLBACK hoàn toàn. |
| Stored Functions & Oracle 23ai SQL BOOLEAN | Tính phí, kiểm tra điều kiện, xếp hạng ví | Tái sử dụng logic trong cả PL/SQL và câu lệnh SELECT SQL thuần; Oracle 23ai hỗ trợ kiểu BOOLEAN trực tiếp trong SQL. |

| Công nghệ / Tính năng | Vai trò trong hệ thống | Lý do lựa chọn |
|---|---|---|
| PL/SQL Packages & Explicit Cursor | Đóng gói nghiệp vụ & đối soát hàng loạt | Tách biệt Specification và Body, chia sẻ biến ngữ cảnh package cho Trigger và duyệt cursor kiểm soát chặt chẽ quá trình đối soát. |
| Compound Trigger (chống ORA-04091) | Kiểm soát hạn mức ngày trên GIAO_DICH | Khắc phục triệt để lỗi Mutating Table (ORA-04091) khi cần tổng hợp nhiều dòng trên chính bảng đang chèn dữ liệu. |
| SELECT ... FOR UPDATE WAIT & SAVEPOINT | Kiểm soát đồng thời & toàn vẹn ACID | Khoá dòng bi quan theo thứ tự tăng dần MA_VI nhằm loại bỏ Deadlock và khôi phục trạng thái nhất quán khi có lỗi. |
| Node.js + Express + oracledb (Thin Mode) | Lớp API kết nối và giao diện Web | Driver chính chủ Oracle chạy thuần JavaScript không cần cài Instant Client, hỗ trợ Connection Pool và OUT Binds. |




*Hình 1.1. Kiến trúc tổng thể 3 tầng hệ thống ví điện tử nội bộ trên Oracle Database 23ai*

## 1.6. Kế hoạch thực hiện và phân công ban đầu của Nhóm 12

Đồ án được thực hiện theo quy trình 8 bước gồm: khảo sát bài toán và chốt phạm vi; phân tích nghiệp vụ và các tình huống ngoại lệ; thiết kế mô hình ER chuẩn 3NF; thiết lập môi trường Docker Oracle 23ai và DBeaver; lập trình các gói PL/SQL, Functions, Triggers; xây dựng API và giao diện Web; kiểm thử và tối ưu hoá Execution Plan; cuối cùng là hoàn thiện báo cáo và chuẩn bị bảo vệ vấn đáp. Bảng chi tiết đóng góp của 3 thành viên (Nguyễn Quốc Huy, Phạm Thành Đức, Nguyễn Thế Vinh) được tổng hợp đầy đủ tại Mục 5.3 của Chương 5.

# CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

## 2.1. Phân tích quy trình nghiệp vụ ví điện tử nội bộ

Hệ thống phục vụ chu trình khép kín của dòng tiền nội bộ trong doanh nghiệp. Dựa trên yêu cầu đề tài, 7 nhóm nghiệp vụ chính được xác định như sau:

- **Nghiệp vụ 1 – Quản lý danh mục ví và trạng thái:** Mỗi nhân viên sở hữu một ví nhân viên (NHAN_VIEN), ngoài ra doanh nghiệp có các ví đối tác dịch vụ nội bộ (DOI_TAC) như Căn tin, Bãi xe, Trung tâm In ấn. Quản trị viên có quyền tạm khoá (TAM_KHOA), mở khoá (HOAT_DONG) hoặc đóng vĩnh viễn (DONG_VINH_VIEN). Mọi thay đổi trạng thái đều phải được Trigger ghi vết lý do và người sửa.
- **Nghiệp vụ 2 – Nạp tiền vào ví (NAP_TIEN):** Ghi nhận nguồn tiền từ quỹ lương hoặc tài khoản ngân hàng liên kết vào ví nhân viên. Yêu cầu ví đích phải đang hoạt động và mã tham chiếu không được trùng lặp.
- **Nghiệp vụ 3 – Rút tiền (RUT_TIEN):** Nhân viên rút tiền từ ví về tài khoản ngân hàng cá nhân. Áp dụng biểu phí cố định cộng tỷ lệ phần trăm (có trần phí tối đa) được tính thông qua Stored Function FN_TINH_PHI_GIAO_DICH. Tổng tiền trừ gồm tiền gốc cộng phí.
- **Nghiệp vụ 4 – Chuyển tiền nội bộ (CHUYEN_NOI_BO):** Chuyển tiền tức thời giữa hai ví nhân viên trong doanh nghiệp. Hệ thống phải khoá bi quan cả hai ví theo thứ tự tăng dần mã ví để ngăn Deadlock, kiểm tra trạng thái hoạt động của cả hai bên, kiểm tra số dư khả dụng và hạn mức ngày.
- **Nghiệp vụ 5 – Thanh toán dịch vụ nội bộ (THANH_TOAN_DV):** Nhân viên thanh toán hoá đơn tại căn tin, bãi giữ xe hoặc trung tâm in ấn. Thủ tục kiểm tra trạng thái hoạt động của đối tác dịch vụ trước khi điều phối sang luồng chuyển tiền lõi.
- **Nghiệp vụ 6 – Hoàn tiền giao dịch (HOAN_TIEN):** Cho phép hoàn trả toàn bộ giá trị giao dịch cho các giao dịch chuyển tiền hoặc thanh toán dịch vụ có trạng thái THANH_CONG. Không cho phép hoàn tiền lần hai đối với giao dịch đã mang trạng thái DA_HOAN_TIEN.

- **Nghiệp vụ 7 – Đối soát sổ cái kế toán kép (SP_DOI_SOAT_HANG_LOAT):** Mỗi biến động số dư đều sinh bút toán GHI_NO / GHI_CO trong CHI_TIET_SO_CAI. Cuối ngày, hệ thống duyệt qua toàn bộ các ví bằng Explicit Cursor để đối chiếu số dư hiện hành trên VI_DIEN_TU với tổng phát sinh Nợ/Có trên sổ cái.

## 2.2. Các tình huống biên và bài toán ngoại lệ đặc thù

Điểm cốt lõi của Đề tài 08 không chỉ là thực hiện thành công giao dịch bình thường, mà là khả năng phát hiện sớm, ngăn chặn triệt để và ghi vết đầy đủ các tình huống bất thường mà ràng buộc CHECK đơn giản không thể tự mình điều phối trọn vẹn. Bảng 2.1 tổng hợp 10 ngoại lệ nghiệp vụ tự định nghĩa và các ngoại lệ định sẵn Oracle được hệ thống xử lý.

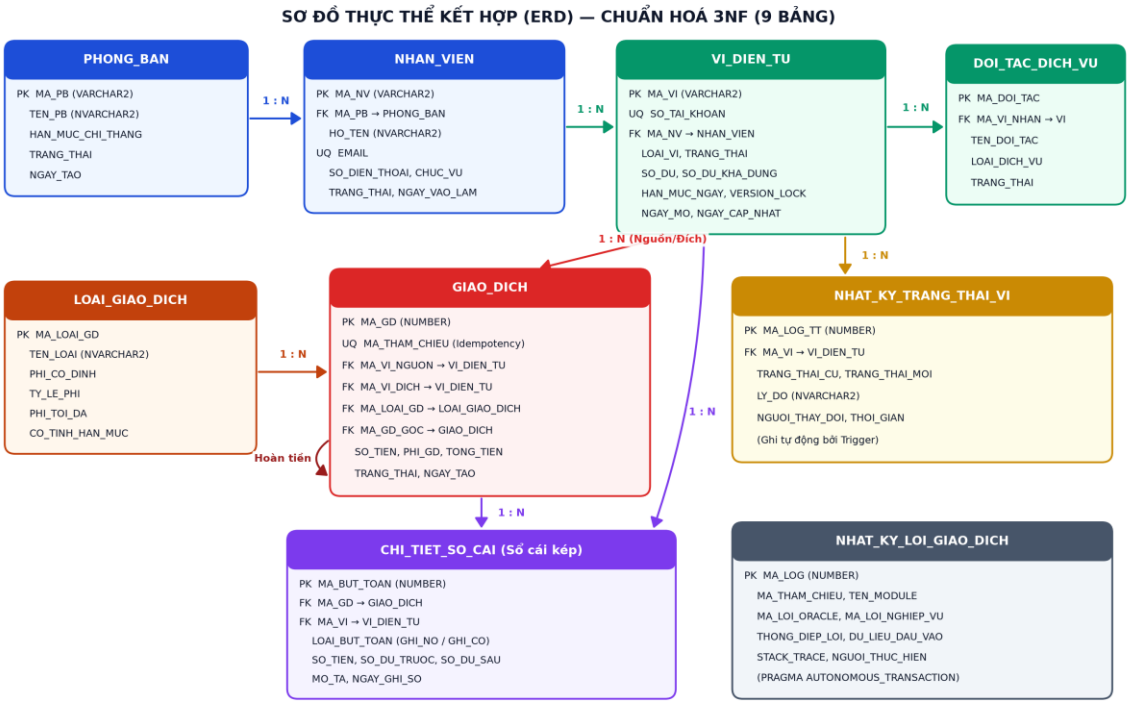
*Bảng 2.1. Danh mục 10 ngoại lệ trọng tâm và cơ chế xử lý trong Oracle 23ai*

| Mã Oracle | Tên Exception (PL/SQL) | Tình huống kích hoạt | Hành động xử lý & Khôi phục |
|---|---|---|---|
| -20001 | e_so_du_khong_du | Số dư khả dụng < (Số tiền + Phí GD) | ROLLBACK về SAVEPOINT, ghi NHAT_KY_LOI_GIAO_DICH. |
| -20002 | e_giao_dich_trung_lap | Trùng MA_THAM_CHIEU hoặc vi phạm UQ_GD_THAM_CHIEU (DUP_VAL_ON_INDEX) | Từ chối xử lý lại giao dịch, giữ nguyên số dư, ghi log trùng. |
| -20003 | e_vi_bi_khoa | Ví nguồn đang TAM_KHOA hoặc DONG_VINH_VIEN | Chặn giao dịch ở cả Stored Procedure và Trigger BEFORE UPDATE. |
| -20004 | e_vi_khong_ton_tai | Bắt NO_DATA_FOUND khi truy vấn VI_DIEN_TU / LOAI_GIAO_DICH / DOI_TAC | Chuyển đổi thành lỗi nghiệp vụ rõ ràng thay vì lộ lỗi hệ thống. |
| -20005 | e_vuot_han_muc_ngay | Tổng chi trong ngày + Giao dịch mới > HAN_MUC_NGAY | Kiểm tra ở PKG_QUAN_LY_VI và chốt chặn lần 2 tại Compound Trigger. |
| -20006 | e_so_tien_khong_hop_le | Số tiền <= 0, NULL hoặc lỗi kiểu VALUE_ERROR | Từ chối ngay đầu thủ tục trước khi cấp khoá dòng. |
| -20007 | e_chuyen_cung_vi | MA_VI_NGUON = MA_VI_DICH khi chuyển tiền | Ngăn tự chuyển cho chính mình gây phình sổ cái giả. |

| Mã Oracle | Tên Exception (PL/SQL) | Tình huống kích hoạt | Hành động xử lý & Khôi phục |
|---|---|---|---|
| -20008 | e_vi_dich_bi_khoa | Ví đích hoặc đối tác dịch vụ đang bị khoá / tạm ngưng | Không ghi có vào ví đang bị phong toả. |
| -20009 | e_khong_the_hoan_tien | Giao dịch gốc không tồn tại, là NAP/RUT hoặc đã hoàn tiền | Chống hoàn tiền lặp nhiều lần gây thất thoát quỹ. |
| -20010 | e_khoa_dong_thoi | Bắt ORA-00054 (NOWAIT) hoặc ORA-30006 (FOR UPDATE WAIT 2 timeout) | Trả thông báo bận đồng thời an toàn để client thử lại. |

## 2.3. Mô hình thực thể kết hợp (ERD) và chuẩn hoá 3NF

Cơ sở dữ liệu được thiết kế với 9 bảng quan hệ, vượt yêu cầu tối thiểu 5 bảng của đồ án và đạt dạng chuẩn 3NF (Third Normal Form): mọi thuộc tính không khoá đều phụ thuộc hàm đầy đủ vào khoá chính và không có phụ thuộc bắc cầu. Cụ thể, thông tin phòng ban (TEN_PB, HAN_MUC_CHI_THANG) được tách khỏi bảng NHAN_VIEN vào PHONG_BAN; thông tin nhân viên được tách khỏi VI_DIEN_TU; biểu phí giao dịch được tách khỏi GIAO_DICH vào bảng danh mục LOAI_GIAO_DICH; và chi tiết phát sinh Nợ/Có của từng ví được tách sang bảng sổ cái kép CHI_TIET_SO_CAI để tránh dư thừa và dễ dàng kiểm toán.




*Hình 2.1. Sơ đồ thực thể kết hợp (ERD) 9 bảng chuẩn 3NF của hệ thống*

## 2.4. Đặc tả chi tiết 9 bảng dữ liệu quan hệ và ràng buộc toàn vẹn

Dưới đây là đặc tả chi tiết cấu trúc cột, kiểu dữ liệu và các ràng buộc toàn vẹn của từng bảng trong schema EWALLET_APP:

*Bảng 2.2. Đặc tả bảng PHONG_BAN và NHAN_VIEN*

| Bảng | Tên cột | Kiểu dữ liệu | Ràng buộc & Mô tả |
|---|---|---|---|
| PHONG_BAN | MA_PB | VARCHAR2(20) | PK – Mã định danh phòng ban (PB_CNTT, PB_KTTC...). |
| PHONG_BAN | TEN_PB | NVARCHAR2(100) | NOT NULL – Tên phòng ban tiếng Việt Unicode. |
| PHONG_BAN | HAN_MUC_CHI_THANG | NUMBER(18,2) | DEFAULT 500000000, CHECK (>= 0) – Hạn mức chi tháng. |
| PHONG_BAN | TRANG_THAI | VARCHAR2(20) | CHECK IN ('HOAT_DONG', 'NGUNG_HOAT_DONG'). |
| NHAN_VIEN | MA_NV | VARCHAR2(20) | PK – Mã nhân viên (NV001..NV010). |
| NHAN_VIEN | HO_TEN | NVARCHAR2(120) | NOT NULL – Họ và tên đầy đủ của nhân viên. |
| NHAN_VIEN | EMAIL | VARCHAR2(120) | NOT NULL, UNIQUE – Email doanh nghiệp duy nhất. |
| NHAN_VIEN | SO_DIEN_THOAI | VARCHAR2(20) | NOT NULL – Số điện thoại liên hệ. |

| Bảng | Tên cột | Kiểu dữ liệu | Ràng buộc & Mô tả |
|---|---|---|---|
| NHAN_VIEN | CHUC_VU | NVARCHAR2(80) | NOT NULL – Chức danh công việc. |
| NHAN_VIEN | MA_PB | VARCHAR2(20) | NOT NULL, FK → PHONG_BAN(MA_PB). |
| NHAN_VIEN | TRANG_THAI | VARCHAR2(20) | CHECK IN ('DANG_LAM_VIEC', 'NGHI_VIEC'). |

*Bảng 2.3. Đặc tả bảng VI_DIEN_TU, DOI_TAC và LOAI_GIAO_DICH*

| Bảng | Tên cột | Kiểu dữ liệu | Ràng buộc & Mô tả |
|---|---|---|---|
| VI_DIEN_TU | MA_VI | VARCHAR2(20) | PK – Mã ví điện tử (VI001, VI_DT_CANTIN...). |
| VI_DIEN_TU | SO_TAI_KHOAN | VARCHAR2(30) | NOT NULL, UNIQUE – Số tài khoản ví nội bộ. |
| VI_DIEN_TU | MA_NV | VARCHAR2(20) | NOT NULL, FK → NHAN_VIEN, UQ(MA_NV, LOAI_VI). |
| VI_DIEN_TU | LOAI_VI | VARCHAR2(20) | CHECK IN ('NHAN_VIEN', 'DOI_TAC', 'HE_THONG'). |
| VI_DIEN_TU | SO_DU | NUMBER(18,2) | DEFAULT 0, CHECK (SO_DU >= 0) – Tổng số dư ví. |
| VI_DIEN_TU | SO_DU_KHA_DUNG | NUMBER(18,2) | CHECK (SO_DU_KHA_DUNG >= 0 AND <= SO_DU). |
| VI_DIEN_TU | HAN_MUC_NGAY | NUMBER(18,2) | DEFAULT 50000000, CHECK (> 0) – Hạn mức chi/ngày. |
| VI_DIEN_TU | TRANG_THAI | VARCHAR2(20) | CHECK IN ('HOAT_DONG', 'TAM_KHOA', 'DONG_VINH_VIEN'). |
| VI_DIEN_TU | VERSION_LOCK | NUMBER(10) | DEFAULT 0 – Đếm phiên bản cập nhật đồng thời. |
| DOI_TAC_DICH_VU | MA_DOI_TAC | VARCHAR2(20) | PK – Mã đối tác (DT_CANTIN, DT_BAIXE, DT_INAN). |
| DOI_TAC_DICH_VU | TEN_DOI_TAC | NVARCHAR2(120) | NOT NULL – Tên đơn vị cung ứng nội bộ. |
| DOI_TAC_DICH_VU | LOAI_DICH_VU | VARCHAR2(40) | CHECK IN ('CAN_TIN', 'BAI_XE', 'IN_AN',...). |
| DOI_TAC_DICH_VU | MA_VI_NHAN | VARCHAR2(20) | NOT NULL, FK → VI_DIEN_TU(MA_VI). |
| LOAI_GIAO_DICH | MA_LOAI_GD | VARCHAR2(30) | PK – NAP_TIEN, RUT_TIEN, CHUYEN_NOI_BO, THANH_TOAN_DV, HOAN_TIEN. |

| Bảng | Tên cột | Kiểu dữ liệu | Ràng buộc & Mô tả |
|---|---|---|---|
| LOAI_GIAO_DICH | PHI_CO_DINH / TY_LE | NUMBER | Quy tắc tính phí; CO_TINH_HAN_MUC IN (0, 1). |

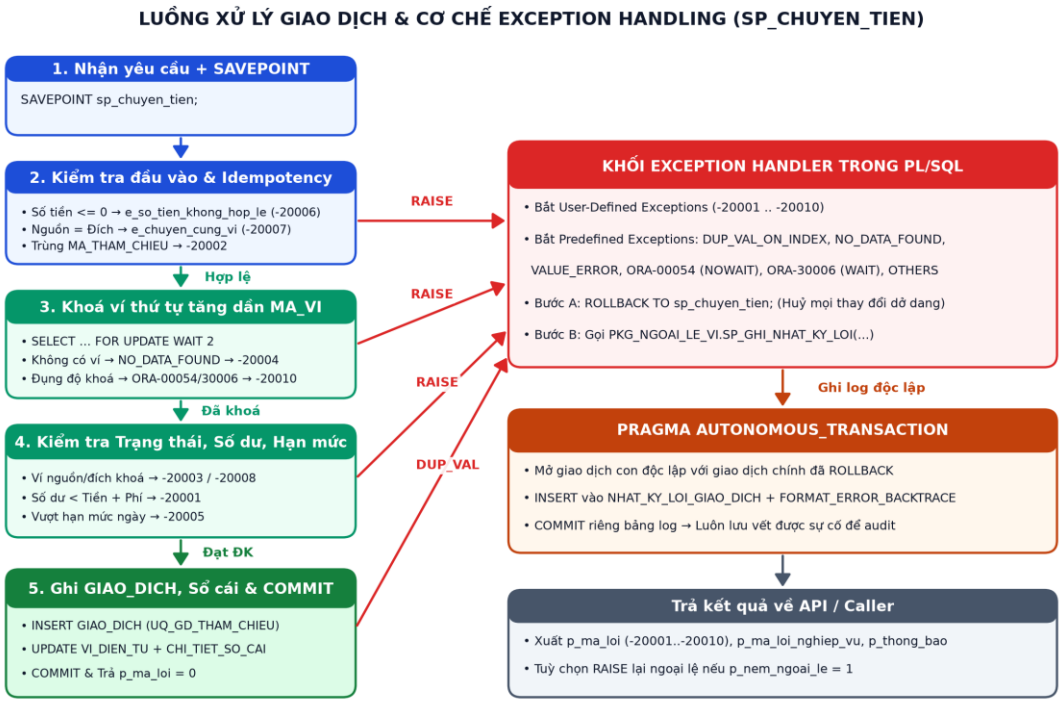
*Bảng 2.4. Đặc tả bảng GIAO_DICH, CHI_TIET_SO_CAI và Audit Logs*

| Bảng | Tên cột | Kiểu dữ liệu | Ràng buộc & Mô tả |
|---|---|---|---|
| GIAO_DICH | MA_GD | NUMBER(18) | PK – Sinh từ SEQ_GIAO_DICH. |
| GIAO_DICH | MA_THAM_CHIEU | VARCHAR2(80) | NOT NULL, UNIQUE – Khoá chống trùng lặp (Idempotency). |
| GIAO_DICH | MA_VI_NGUON / DICH | VARCHAR2(20) | FK → VI_DIEN_TU(MA_VI) (NULL với nguồn nạp / đích rút). |
| GIAO_DICH | MA_LOAI_GD / GOC | VARCHAR2 / NUMBER | FK → LOAI_GIAO_DICH; FK tự tham chiếu khi hoàn tiền. |
| GIAO_DICH | SO_TIEN / PHI / TONG | NUMBER(18,2) | CHECK (SO_TIEN > 0, PHI_GD >= 0, TONG_TIEN > 0). |
| GIAO_DICH | TRANG_THAI | VARCHAR2(20) | CHECK IN ('CHO_XU_LY', 'THANH_CONG', 'THAT_BAI', 'DA_HOAN_TIEN'). |
| CHI_TIET_SO_CAI | MA_BUT_TOAN | NUMBER(18) | PK – Sinh từ SEQ_SO_CAI. |
| CHI_TIET_SO_CAI | MA_GD / MA_VI | NUMBER / VARCHAR2 | FK → GIAO_DICH, FK → VI_DIEN_TU. |
| CHI_TIET_SO_CAI | LOAI_BUT_TOAN | VARCHAR2(10) | CHECK IN ('GHI_NO', 'GHI_CO'). |
| CHI_TIET_SO_CAI | SO_DU_TRUOC / SAU | NUMBER(18,2) | CHECK (>= 0) – Lưu ảnh chụp số dư trước và sau bút toán. |
| NHAT_KY_LOI_GD | MA_LOG / MA_LOI | NUMBER | PK; Mã lỗi Oracle (-20001..-20010, -1, -54...) + STACK_TRACE. |
| NHAT_KY_TT_VI | MA_LOG_TT / MA_VI | NUMBER / VARCHAR2 | PK; FK → VI_DIEN_TU; Ghi tự động khi khoá/mở ví. |

## 2.5. Thiết kế kiến trúc xử lý ngoại lệ đa tầng và chống tranh chấp đồng thời

Trong hệ thống thanh toán ví điện tử, các sự cố thường xảy ra theo ba nhóm: (1) Lỗi nghiệp vụ do dữ liệu đầu vào hoặc trạng thái tài khoản; (2) Lỗi tranh chấp đồng thời (Concurrency) khi hai giao dịch cùng chạm vào một ví hoặc cùng gửi một mã tham chiếu tại cùng một mili-giây; (3) Lỗi mất dữ liệu nhật ký khi giao dịch bị ROLLBACK. Để giải quyết triệt để, nhóm thiết kế kiến trúc phòng vệ 3 lớp:

- **Lớp 1 – Idempotency Key và Ràng buộc Duy nhất:** Mọi yêu cầu giao dịch đều mang MA_THAM_CHIEU duy nhất. Thủ tục kiểm tra trước bằng SELECT, đồng thời đặt ràng buộc UNIQUE (UQ_GD_THAM_CHIEU) trên cột GIAO_DICH. MA_THAM_CHIEU. Nếu hai luồng song song vượt qua bước SELECT cùng lúc, Oracle sẽ ném ngoại lệ định sẵn DUP_VAL_ON_INDEX cho luồng đến sau và khối EXCEPTION lập tức ánh xạ về mã -20002.
- **Lớp 2 – Khoá bi quan theo thứ tự xác định chống Deadlock:** Khi chuyển tiền giữa ví A và ví B, nếu giao dịch 1 khoá A rồi chờ B trong khi giao dịch 2 khoá B rồi chờ A thì sẽ xuất hiện Deadlock (ORA-00060). Giải pháp của hệ thống là luôn so sánh chuỗi MA_VI để khoá ví có mã nhỏ hơn trước (IF p_ma_vi_nguon < p_ma_vi_dich), kết hợp mệnh đề FOR UPDATE WAIT 2. Nếu quá 2 giây chưa lấy được khoá, Oracle ném ORA-30006 và được bắt gọn thành lỗi -20010.
- **Lớp 3 – SAVEPOINT và PRAGMA AUTONOMOUS_TRANSACTION:** Đầu mỗi giao dịch thiết lập SAVEPOINT. Khi bất kỳ ngoại lệ nào xảy ra, khối EXCEPTION thực hiện ROLLBACK TO SAVEPOINT để bảo đảm tính Atomicity (không bao giờ có chuyện trừ tiền ví nguồn mà chưa cộng ví đích), sau đó gọi PKG_NGOAI_LE_VI.SP_GHI_NHAT_KY_LOI chạy trong một Autonomous Transaction độc lập để COMMIT bản ghi lỗi vào NHAT_KY_LOI_GIAO_DICH kèm theo dấu vết ngăn xếp DBMS_UTILITY.FORMAT_ERROR_BACKTRACE.




*Hình 2.2. Lưu đồ luồng xử lý giao dịch và kiến trúc Exception Handling đa tầng*

# CHƯƠNG 3. CÀI ĐẶT HỆ THỐNG VÀ TRIỂN KHAI

## 3.1. Triển khai Oracle Database 23ai trên Docker và cấu hình DBeaver

Toàn bộ hạ tầng cơ sở dữ liệu được đóng gói bằng Docker Compose sử dụng image chính thức Oracle Database 23ai Free (container-registry.oracle.com/database/free:latest). Điểm quan trọng khi triển khai cho bài toán tiếng Việt là cấu hình biến môi trường ORACLE_CHARACTERSET=AL32UTF8 và NLS_LANG=AMERICAN_AMERICA.AL32UTF8 để toàn bộ dữ liệu NVARCHAR2 và thông báo lỗi tiếng Việt không bị lỗi font khi chạy qua sqlplus.

**Đoạn mã 3.1. Cấu hình docker-compose.yml khởi tạo Oracle Database 23ai Free kèm Healthcheck**

```
services:
  oracle23ai:
    image: container-registry.oracle.com/database/free:latest
    container_name: oracle23ai_ewallet
    ports:
      - "1521:1521"
      - "5500:5500"
    environment:
      ORACLE_PWD: ${ORACLE_PWD:-Oracle23ai}
      ORACLE_CHARACTERSET: AL32UTF8
      NLS_LANG: AMERICAN_AMERICA.AL32UTF8
    volumes:
      - oracle23ai_data:/opt/oracle/oradata
      - ./sql:/opt/oracle/user_sql:ro
      - ./docker/init:/opt/oracle/scripts/setup:ro
    healthcheck:
      test: ["CMD-SHELL", "echo 'SELECT 1 FROM DUAL;' | sqlplus -s
sys/Oracle23ai@//localhost:1521/FREEPDB1 as sysdba | grep -q 1"]
      interval: 15s
      retries: 20
```

Toàn bộ 11 tệp kịch bản CSDL được chuẩn bị với hai phiên bản đồng bộ: thư mục sql/ (sử dụng dấu gạch chéo / trên dòng riêng để phân tách các khối PL/SQL khi chạy tự động qua SQL*Plus hoặc script PowerShell trên Docker) và thư mục sql_dbeaver/ (đã loại bỏ các dấu / đứng riêng để tránh lỗi ORA-00900: invalid SQL statement khi chạy hàng loạt bằng tính năng Execute SQL Script – Alt+X trên DBeaver 26.x). Khi kết nối DBeaver, người dùng cần tạo kết nối trực tiếp bằng đúng tài khoản sở hữu schema

EWALLET_APP / Ewallet2026 tới Host=localhost, Port=1521, Service Name=FREEPDB1 để hiển thị đầy đủ 9 bảng và thực thi các kịch bản kiểm thử.

## 3.2. Cài đặt Package quản lý ngoại lệ PKG_NGOAI_LE_VI

Package PKG_NGOAI_LE_VI đóng vai trò trung tâm chuẩn hoá toàn bộ mã lỗi của hệ thống. Thay vì dùng các câu lệnh RAISE_APPLICATION_ERROR rời rạc ở nhiều nơi gây khó bảo trì, mọi ngoại lệ được khai báo tập trung trong Package Specification và liên kết mã lỗi Oracle thông qua chỉ thị biên dịch PRAGMA EXCEPTION_INIT.

**Đoạn mã 3.2. Trích đoạn khai báo ngoại lệ và thủ tục ghi log độc lập trong PKG_NGOAI_LE_VI**

```
CREATE OR REPLACE PACKAGE PKG_NGOAI_LE_VI AS
    e_so_du_khong_du        EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_so_du_khong_du, -20001);
    e_giao_dich_trung_lap   EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_giao_dich_trung_lap, -20002);
    e_vi_bi_khoa            EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_vi_bi_khoa, -20003);
    e_ora_resource_busy     EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_ora_resource_busy, -54);
    e_ora_wait_timeout      EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_ora_wait_timeout, -30006);
    ...
END PKG_NGOAI_LE_VI;
/

PROCEDURE SP_GHI_NHAT_KY_LOI( ... ) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO NHAT_KY_LOI_GIAO_DICH (
        MA_LOG, MA_THAM_CHIEU, TEN_MODULE, MA_LOI_ORACLE,
        MA_LOI_NGHIEP_VU, THONG_DIEP_LOI, DU_LIEU_DAU_VAO, STACK_TRACE
    ) VALUES (
        SEQ_LOG_LOI.NEXTVAL, p_ma_tham_chieu, p_ten_module, p_ma_loi_oracle,
        p_ma_loi_nghiep_vu, p_thong_diep_loi, p_du_lieu_dau_vao, p_stack_trace
    );
    COMMIT; -- Chi commit giao dich ghi log doc lap, khong anh huong ROLLBACK chinh
EXCEPTION
    WHEN OTHERS THEN ROLLBACK;
END SP_GHI_NHAT_KY_LOI;
```

Cơ chế PRAGMA AUTONOMOUS_TRANSACTION đặc biệt quan trọng trong nghiệp vụ ví điện tử: khi giao dịch chuyển tiền gặp lỗi -20001 và bị ROLLBACK TO sp_chuyen_tien, bản ghi trong bảng NHAT_KY_LOI_GIAO_DICH vẫn được

COMMIT thành công nhờ phiên giao dịch con độc lập, giúp đội ngũ vận hành truy vết được thời điểm, tham số đầu vào và dòng mã gây lỗi.

### 3.3. Cài đặt các Stored Functions và khả năng gọi trong câu lệnh SQL

Tệp sql/06_functions.sql cài đặt 5 Stored Functions phục vụ tính toán và kiểm tra điều kiện. Một câu hỏi trọng tâm trong vấn đáp PL/SQL là: "Hàm (Function) có thể gọi được trong câu lệnh SQL thông thường (SELECT) hay không? Vì sao?". Theo quy tắc của Oracle, một hàm gọi được trong câu lệnh SQL thuần phải thoả mãn các điều kiện: (1) Không thực hiện lệnh DML (INSERT/UPDATE/DELETE) hay điều khiển giao dịch (COMMIT/ROLLBACK) lên các bảng CSDL; (2) Chỉ sử dụng các tham số chế độ IN (không dùng OUT hoặc IN OUT); (3) Kiểu dữ liệu tham số và giá trị trả về phải là kiểu được SQL hỗ trợ.

Đặc biệt, trên các phiên bản Oracle 19c trở về trước, kiểu BOOLEAN chỉ tồn tại trong PL/SQL nên hàm trả về BOOLEAN không thể gọi trực tiếp trong câu lệnh SELECT. Tuy nhiên, từ Oracle Database 23ai, Oracle đã hỗ trợ kiểu dữ liệu BOOLEAN gốc (native SQL BOOLEAN), cho phép gọi trực tiếp hàm FN_KIEM_TRA_VI_HOAT_DONG (nhận tham số MA_VI, trả về BOOLEAN) bên trong truy vấn SELECT để phục vụ giao diện và kiểm tra nhanh.

**Đoạn mã 3.3. Trích đoạn FN_TINH_PHI_GIAO_DICH và hàm BOOLEAN gốc SQL trên Oracle 23ai**

```
CREATE OR REPLACE FUNCTION FN_TINH_PHI_GIAO_DICH(
    p_ma_loai_gd IN VARCHAR2,
    p_so_tien    IN NUMBER
) RETURN NUMBER IS
    v_phi_co_dinh LOAI_GIAO_DICH.PHI_CO_DINH%TYPE;
    v_ty_le_phi   LOAI_GIAO_DICH.TY_LE_PHI%TYPE;
    v_phi_toi_da  LOAI_GIAO_DICH.PHI_TOI_DA%TYPE;
    v_phi_tinh    NUMBER(18, 2);
BEGIN
    IF p_so_tien IS NULL OR p_so_tien <= 0 THEN
        RAISE PKG_NGOAI_LE_VI.e_so_tien_khong_hop_le;
    END IF;
    SELECT PHI_CO_DINH, TY_LE_PHI, PHI_TOI_DA
      INTO v_phi_co_dinh, v_ty_le_phi, v_phi_toi_da
      FROM LOAI_GIAO_DICH WHERE MA_LOAI_GD = p_ma_loai_gd;
    v_phi_tinh := ROUND(v_phi_co_dinh + (p_so_tien * v_ty_le_phi), 2);
    IF v_phi_toi_da > 0 AND v_phi_tinh > v_phi_toi_da THEN
        v_phi_tinh := v_phi_toi_da;
    END IF;
    RETURN v_phi_tinh;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20004, 'Khong tim thay loai giao dich: ' || p_ma_loai_gd);
    WHEN VALUE_ERROR THEN
        RAISE_APPLICATION_ERROR(-20006, 'Gia tri so tien tinh phi khong hop le.');
END FN_TINH_PHI_GIAO_DICH;
/

CREATE OR REPLACE FUNCTION FN_KIEM_TRA_VI_HOAT_DONG(
    p_ma_vi IN VARCHAR2
) RETURN BOOLEAN IS
    v_trang_thai VI_DIEN_TU.TRANG_THAI%TYPE;
BEGIN
    SELECT TRANG_THAI INTO v_trang_thai FROM VI_DIEN_TU WHERE MA_VI = p_ma_vi;
    RETURN (v_trang_thai = 'HOAT_DONG');
EXCEPTION
    WHEN NO_DATA_FOUND OR TOO_MANY_ROWS THEN RETURN FALSE;
END FN_KIEM_TRA_VI_HOAT_DONG;
```

## 3.4. Cài đặt Package nghiệp vụ cốt lõi PKG_QUAN_LY_VI

Package PKG_QUAN_LY_VI gom toàn bộ 7 thủ tục nghiệp vụ vào một đơn vị đóng gói thống nhất: SP_NAP_TIEN, SP_RUT_TIEN, SP_CHUYEN_TIEN, SP_THANH_TOAN_DICH_VU, SP_HOAN_TIEN, SP_KHOA_MO_VI và SP_DOI_SOAT_HANG_LOAT. Việc đặt chung trong Package giúp Oracle biên dịch và nạp một lần vào Shared Pool, tái sử dụng biến trạng thái phiên (như g_ly_do_doi_tt) và ẩn các hàm phụ trợ nội bộ (như fn_map_err_code) khỏi người dùng bên ngoài.

**Đoạn mã 3.4. Trích đoạn khoá thứ tự tăng dần chống Deadlock và khối bắt lỗi trong SP_CHUYEN_TIEN**

```
SAVEPOINT sp_chuyen_tien;
...
-- Khoa bi quan theo thu tu tang dan MA_VI nham loai bo hoan toan Deadlock
IF p_ma_vi_nguon < p_ma_vi_dich THEN
    SELECT SO_DU, SO_DU_KHA_DUNG, HAN_MUC_NGAY, TRANG_THAI
      INTO v_src_du, v_src_kd, v_src_hm, v_src_tt
      FROM VI_DIEN_TU WHERE MA_VI = p_ma_vi_nguon FOR UPDATE WAIT 2;
    SELECT SO_DU, TRANG_THAI INTO v_dst_du, v_dst_tt
      FROM VI_DIEN_TU WHERE MA_VI = p_ma_vi_dich FOR UPDATE WAIT 2;
ELSE
    SELECT SO_DU, TRANG_THAI INTO v_dst_du, v_dst_tt
      FROM VI_DIEN_TU WHERE MA_VI = p_ma_vi_dich FOR UPDATE WAIT 2;
    SELECT SO_DU, SO_DU_KHA_DUNG, HAN_MUC_NGAY, TRANG_THAI
      INTO v_src_du, v_src_kd, v_src_hm, v_src_tt
      FROM VI_DIEN_TU WHERE MA_VI = p_ma_vi_nguon FOR UPDATE WAIT 2;
END IF;
...
EXCEPTION
    WHEN PKG_NGOAI_LE_VI.e_giao_dich_trung_lap OR DUP_VAL_ON_INDEX THEN
        ROLLBACK TO sp_chuyen_tien;
        p_ma_loi := -20002;
        p_ma_loi_nghiep_vu := PKG_NGOAI_LE_VI.C_ERR_GIAO_DICH_TRUNG;
        p_thong_bao := PKG_NGOAI_LE_VI.FN_LAY_THONG_DIEP_LOI(-20002);
        PKG_NGOAI_LE_VI.SP_GHI_NHAT_KY_LOI(...);
        IF p_nem_ngoai_le = 1 THEN RAISE PKG_NGOAI_LE_VI.e_giao_dich_trung_lap; END
IF;
```

Đối với nghiệp vụ đối soát cuối ngày (SP_DOI_SOAT_HANG_LOAT), nhóm sử dụng tường minh Explicit Cursor (OPEN – FETCH – EXIT WHEN %NOTFOUND – CLOSE) thay vì Cursor FOR Loop thông thường. Việc dùng Explicit Cursor cho phép kiểm soát chặt chẽ vòng đời con trỏ, đóng con trỏ an toàn trong khối EXCEPTION (IF c_vi_doi_soat%ISOPEN THEN CLOSE ...) và ghi log cảnh báo chi tiết cho từng ví nếu phát hiện độ lệch với sổ cái.

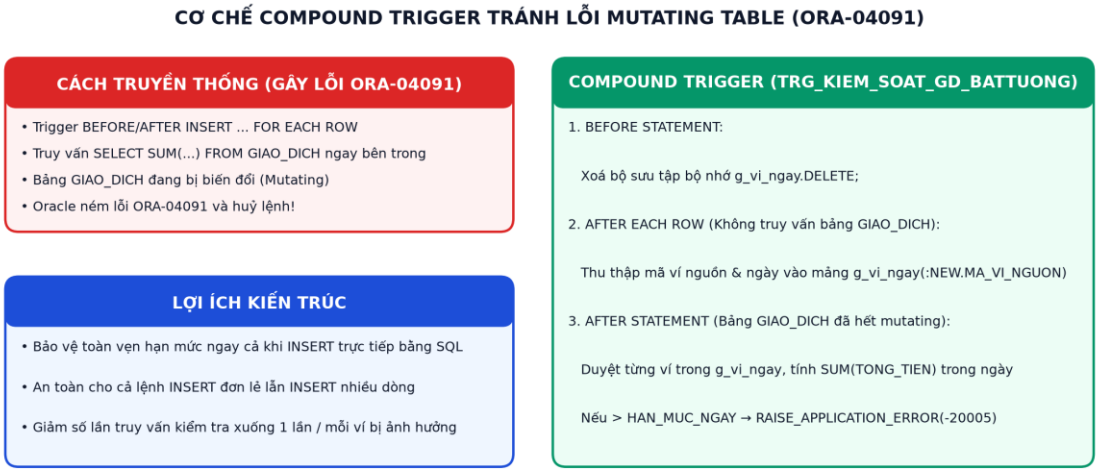
**Đoạn mã 3.5. Trích đoạn Explicit Cursor đối soát sổ cái trong SP_DOI_SOAT_HANG_LOAT**

```
OPEN c_vi_doi_soat;
LOOP
    FETCH c_vi_doi_soat INTO v_rec;
    EXIT WHEN c_vi_doi_soat%NOTFOUND;
    p_tong_vi_kiem_tra := p_tong_vi_kiem_tra + 1;
    IF v_rec.SO_DU <> v_rec.SO_DU_SO_CAI THEN
        p_so_vi_lech := p_so_vi_lech + 1;
        PKG_NGOAI_LE_VI.SP_GHI_NHAT_KY_LOI(...);
    END IF;
END LOOP;
CLOSE c_vi_doi_soat;
```

## 3.5. Cài đặt Triggers bảo vệ số dư và Compound Trigger chống Mutating Table

Hệ thống triển khai 3 Trigger trong tệp sql/08_triggers.sql nhằm bảo đảm nguyên tắc "CSDL tự bảo vệ mình" ngay cả khi có lệnh SQL cập nhật trực tiếp mà không đi qua Package API:

- **TRG_KIEM_TRA_SO_DU_VI (BEFORE UPDATE):** Chặn mọi hành vi sửa cột SO_DU hoặc SO_DU_KHA_DUNG khi ví đang ở trạng thái TAM_KHOA hoặc DONG_VINH_VIEN (-20003), và chặn số dư âm (-20001). Trigger chỉ đọc :OLD và :NEW nên không gây lỗi Mutating Table.
- **TRG_NHAT_KY_TRANG_THAI_VI (AFTER UPDATE):** Tự động ghi bản ghi vào NHAT_KY_TRANG_THAI_VI mỗi khi cột TRANG_THAI thay đổi, đọc lý do và người thực hiện từ biến ngữ cảnh của PKG_QUAN_LY_VI (hoặc gán mặc định USER nếu sửa trực tiếp từ SQL).
- **TRG_KIEM_SOAT_GD_BATTUONG (COMPOUND TRIGGER FOR INSERT ON GIAO_DICH):** Nếu dùng row-level trigger thông thường để truy vấn SELECT SUM(TONG_TIEN) FROM GIAO_DICH kiểm tra hạn mức ngày, Oracle sẽ báo lỗi ORA-04091 (table is mutating). Kiến trúc Compound Trigger cho phép tối đa 4 pha thời điểm; trong đó trigger này triển khai 3 pha cần thiết: BEFORE STATEMENT xoá bộ nhớ đệm; AFTER EACH ROW lưu lại mã ví nguồn và ngày vào mảng PL/SQL g_vi_ngay; AFTER STATEMENT mới thực hiện truy vấn tổng hợp trên GIAO_DICH khi bảng đã hết trạng thái biến đổi.




*Hình 3.1. Cơ chế Compound Trigger ngăn ngừa lỗi Mutating Table (ORA-04091)*

## 3.6. Xây dựng lớp API Backend và Giao diện Web minh hoạ

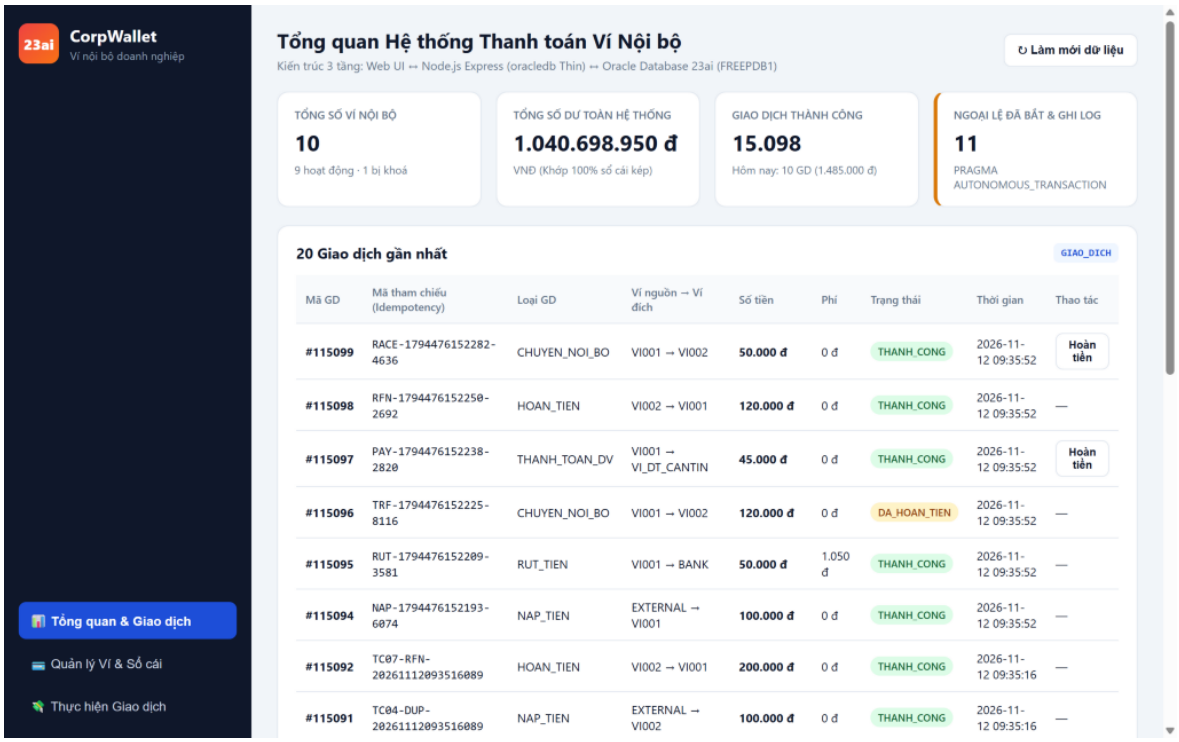
Đáp ứng yêu cầu bắt buộc tại Mục 3.3 của đề cương đồ án, nhóm xây dựng ứng dụng Web 3 tầng hoàn chỉnh trong thư mục backend/. Tầng API sử dụng Node.js + Express kết hợp driver chính chủ oracledb (chế độ Thin Mode kết nối trực tiếp TCP đến cổng 1521/FREEPDB1 thông qua Connection Pool từ 2 đến 10 kết nối).

*Bảng 3.1. Danh sách các Endpoint REST API và ánh xạ tới đối tượng Oracle 23ai*

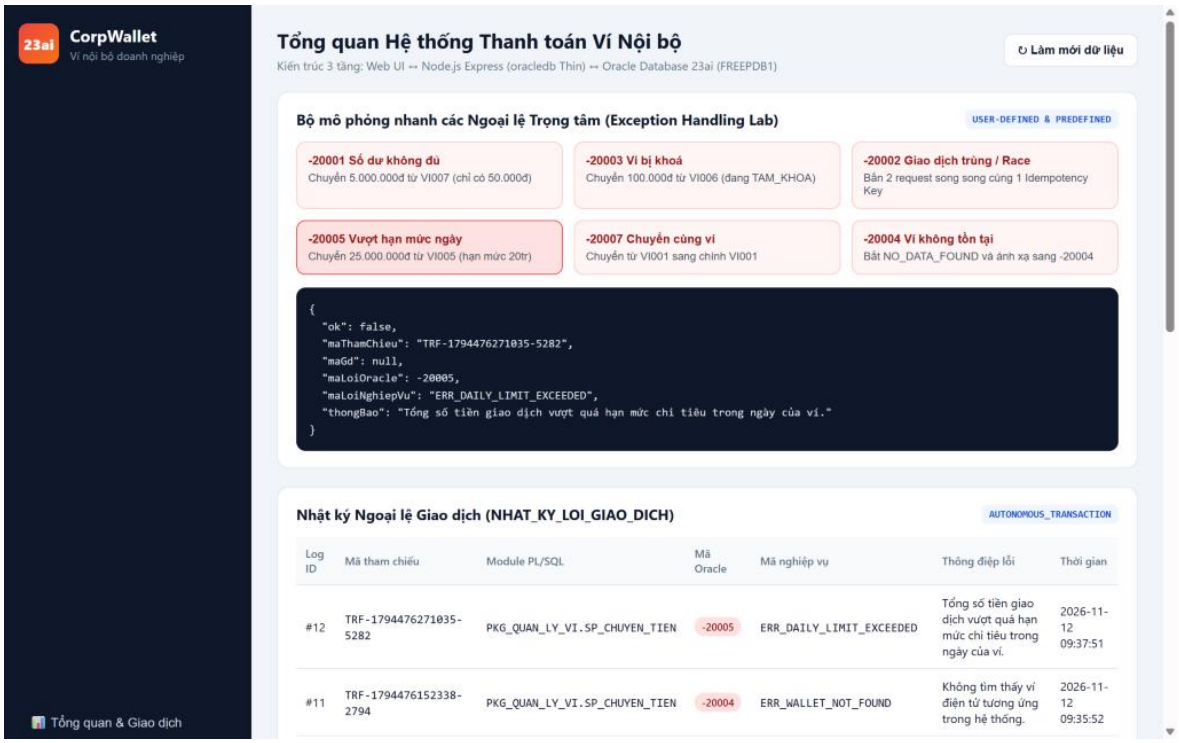
| HTTP Endpoint | Đối tượng Oracle 23ai được gọi | Chức năng nghiệp vụ |
|---|---|---|
| GET /api/health | V$VERSION, SYS_CONTEXT | Kiểm tra kết nối Oracle 23ai và tên PDB đang hoạt động. |
| GET /api/dashboard | VI_DIEN_TU, GIAO_DICH, NHAT_KY_LOI_GIAO_DICH | Thống kê tổng quan số ví, tổng số dư, 20 GD và 15 log lỗi mới nhất. |
| GET /api/wallets | VW_TONG_HOP_VI_ NHAN_VIEN + FN_* | Lấy danh sách ví kèm tổng chi trong ngày và xếp hạng ví từ SQL Function. |
| GET /api/wallets/:maVi/details | CHI_TIET_SO_CAI, NHAT_KY_TRANG_THAI_VI | Xem sổ cái kế toán kép và lịch sử khoá/mở ví do Trigger ghi. |
| GET /api/fee | FN_TINH_PHI_GIAO_DICH | Tính trước phí giao dịch ngay khi người dùng nhập số tiền trên Web. |
| POST /api/transactions/deposit | PKG_QUAN_LY_VI. SP_NAP_TIEN | Gọi thủ tục nạp tiền, nhận OUT Binds (maGd, maLoi, thongBao). |
| POST /api/transactions/withdraw | PKG_QUAN_LY_VI. SP_RUT_TIEN | Gọi thủ tục rút tiền có tính phí và kiểm tra hạn mức ngày. |
| POST /api/transactions/transfer | PKG_QUAN_LY_VI. SP_CHUYEN_TIEN | Gọi thủ tục chuyển tiền nội bộ, hỗ trợ truyền lại Idempotency Key. |

| HTTP Endpoint | Đối tượng Oracle 23ai được gọi | Chức năng nghiệp vụ |
|---|---|---|
| POST /api/transactions/pay-service | PKG_QUAN_LY_VI. SP_THANH_TOAN_DICH_VU | Thanh toán tại Căn tin / Bãi xe / Trung tâm In ấn. |
| POST /api/transactions/refund | PKG_QUAN_LY_VI. SP_HOAN_TIEN | Hoàn tiền cho giao dịch đã thành công ngay trên Dashboard. |
| POST /api/wallets/:maVi/status | PKG_QUAN_LY_VI. SP_KHOA_MO_VI | Khoá hoặc mở khoá ví, kích hoạt Trigger ghi audit log. |
| POST /api/reconcile | PKG_QUAN_LY_VI. SP_DOI_SOAT_HANG_LOAT | Chạy Explicit Cursor đối soát toàn bộ ví và sổ cái. |
| POST /api/test-concurrent-duplicate | Promise.all → SP_CHUYEN_TIEN | Bắn 2 request song song cùng 1 mã tham chiếu để kiểm thử Race Condition. |

Luồng dữ liệu khép kín khi người dùng thực hiện chuyển tiền trên giao diện Web diễn ra như sau: (1) Trình duyệt tại backend/public/app.js thu thập dữ liệu form (ví nguồn, ví đích, số tiền, mã tham chiếu) và gửi HTTP POST JSON tới /api/transactions/transfer; (2) Express server trong server.js lấy một kết nối từ Oracle Connection Pool, ánh xạ các biến đầu vào và khai báo 4 biến oracledb.BIND_OUT (p_ma_gd, p_ma_loi, p_ma_loi_nghiep_vu, p_thong_bao) để gọi khối PL/SQL PKG_QUAN_LY_VI.SP_CHUYEN_TIEN; (3) Oracle 23ai thực thi logic nghiệp vụ, kích hoạt Compound Trigger TRG_KIEM_SOAT_GD_BATTUONG và trả kết quả về OUT Binds; (4) Nếu p_ma_loi = 0, API trả HTTP 200 kèm mã giao dịch mới, ngược lại trả HTTP 400 kèm mã lỗi nghiệp vụ và thông điệp tiếng Việt để giao diện hiển thị Toast thông báo và cập nhật ngay bảng Nhật ký ngoại lệ.




*Hình 3.2. Giao diện Dashboard thực tế kết nối Oracle Database 23ai Free (FREEPDB1)*




*Hình 3.3. Kiểm thử nhanh ngoại lệ (-20005) và Nhật ký lỗi trên giao diện Web*

# CHƯƠNG 4. KIỂM THỬ VÀ TỐI ƯU HOÁ

## 4.1. Môi trường và tập dữ liệu kiểm thử

Tập dữ liệu kiểm thử được nạp thực tế trên container Oracle Database 23ai Free (phiên bản 23.7.0.0, PDB FREEPDB1) từ tệp sql/04_seed_data.sql thông qua kỹ thuật FORALL Bulk Insert với kiểu %ROWTYPE (tránh lỗi PLS-00436). Quy mô dữ liệu thực nghiệm gồm: 5 phòng ban, 10 nhân viên, 10 ví điện tử (trong đó ví VI006 được đặt sẵn trạng thái TAM_KHOA, ví VI007 chỉ có số dư khởi tạo 50.000 VNĐ và ví VI005 có số dư khả dụng vượt hạn mức ngày 20.000.000 VNĐ), 3 đối tác dịch vụ nội bộ, 5 loại giao dịch, hơn 15.000 giao dịch lịch sử phân bố trong 180 ngày gần nhất và hơn 30.000 bút toán sổ cái kế toán kép cân bằng 100% với số dư của từng ví. Sau khi nạp dữ liệu, DBMS_STATS.GATHER_SCHEMA_STATS được gọi để cập nhật thống kê chính xác cho Oracle Cost-Based Optimizer (CBO).

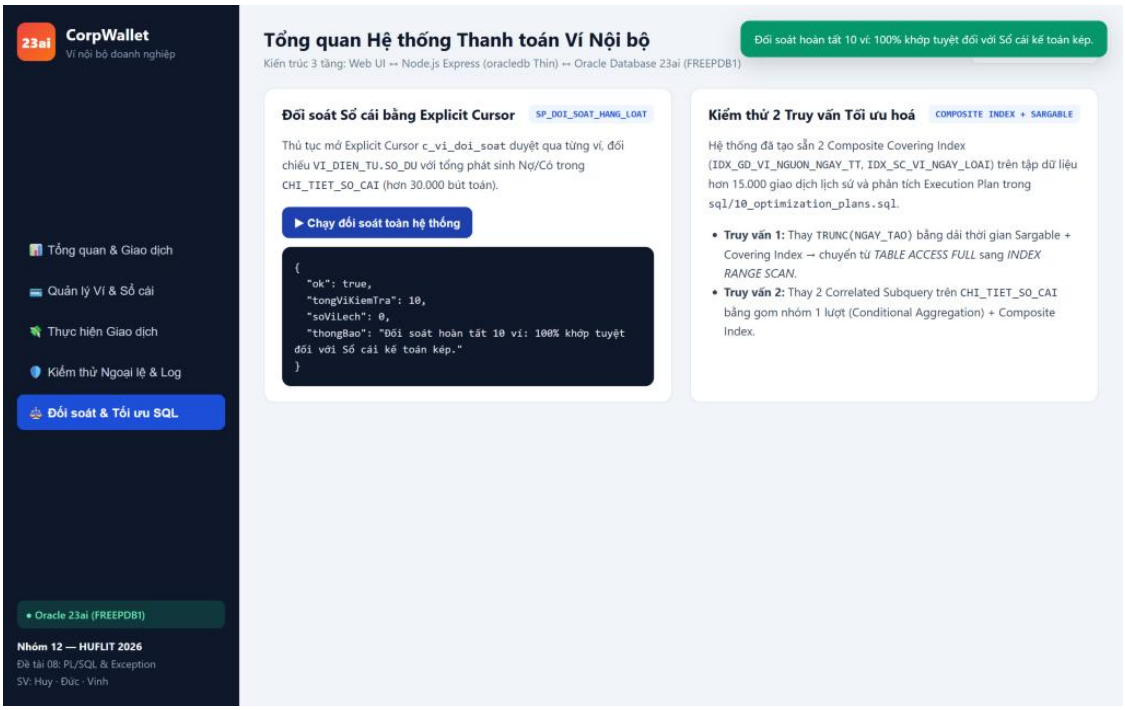
## 4.2. Bảng kịch bản kiểm thử và kết quả xử lý ngoại lệ

Bộ kiểm thử tại sql/09_test_scenarios.sql và Tab "Kiểm thử Ngoại lệ & Log" trên giao diện Web thực hiện 9 kịch bản trọng tâm, bao phủ cả luồng chuẩn (Happy Path), các trường hợp biên/ngoại lệ, Compound Trigger và tranh chấp đồng thời:

*Bảng 4.1. Tổng hợp 9 kịch bản kiểm thử chức năng, ngoại lệ và kết quả xác minh thực tế*

| Mã TC | Mô tả kịch bản kiểm thử | Cơ chế / Ngoại lệ kiểm tra | Kết quả xác minh thực tế trên Oracle 23ai | Trạng thái |
|---|---|---|---|---|
| TC01 | Nạp tiền hợp lệ vào VI001 (+500.000đ) | SP_NAP_TIEN | p_ma_loi = 0; VI001 tăng đúng 500.000đ; sinh 1 GD NAP_TIEN và 1 bút toán GHI_CO khớp số dư trước/sau. | Đạt |
| TC02 | Chuyển tiền nội bộ hợp lệ VI001 → VI002 (200.000đ) | SP_CHUYEN_TIEN | p_ma_loi = 0; VI001 giảm 200.000đ (GHI_NO), VI002 tăng 200.000đ (GHI_CO); tổng số dư hệ thống không đổi. | Đạt |
| TC03 | Ngoại lệ số dư không đủ (VI007 số dư 50.000đ chuyển 5.000.000đ) | e_so_du_khong_du (-20001) | ROLLBACK TO sp_chuyen_tien; số dư VI007 giữ nguyên 50.000đ; NHAT_KY_LOI_GIAO_DICH ghi nhận bản ghi -20001. | Đạt |
| TC04 | Ngoại lệ giao dịch trùng lặp (Gửi 2 lần hoặc 2 luồng song song cùng MA_THAM_CHIEU) | e_giao_dich_trung_lap (-20002) / DUP_VAL_ON_INDEX | Thử nghiệm song song Promise.all trên API trả chính xác 1 luồng mã 0 và 1 luồng bị chặn mã -20002. | Đạt |

| Mã TC | Mô tả kịch bản kiểm thử | Cơ chế / Ngoại lệ kiểm tra | Kết quả xác minh thực tế trên Oracle 23ai | Trạng thái |
|---|---|---|---|---|
| TC05 | Ngoại lệ ví bị khoá (Chuyển tiền từ VI006 đang TAM_KHOA) | e_vi_bi_khoa (-20003) | Thủ tục phát hiện TRANG_THAI <> 'HOAT_DONG', ném -20003 và ghi log; thử UPDATE trực tiếp cũng bị Trigger chặn. | Đạt |
| TC06 | Ngoại lệ vượt hạn mức ngày (VI005 số dư > 35tr, hạn mức 20tr, chuyển 25tr) | e_vuot_han_muc_ngay (-20005) | Trả mã -20005 (ERR_DAILY_LIMIT_ EXCEEDED) cả trên SQL script lẫn Exception Lab. | Đạt |
| TC07 | Hoàn tiền GD TC02 và thử hoàn tiền tiếp lần 2 | SP_HOAN_TIEN & e_khong_the_hoan_tien (-20009) | Lần 1 tạo GD HOAN_TIEN và đổi GD gốc sang DA_HOAN_TIEN; lần 2 bị chặn mã -20009 (ERR_REFUND_NOT_ ALLOWED). | Đạt |
| TC08 | Đối soát toàn bộ hệ thống & gọi SQL BOOLEAN Oracle 23ai | SP_DOI_SOAT_ HANG_LOAT & FN_KIEM_TRA_VI_ HOAT_DONG | Explicit Cursor duyệt toàn bộ 10 ví cho kết quả 0 ví lệch (khớp 100% với hơn 30.000 bút toán sổ cái); SQL trả TRUE/FALSE. | Đạt |
| TC09 | Kiểm thử trực tiếp Compound Trigger trên bảng GIAO_DICH | TRG_KIEM_SOAT_GD_ BATTUONG (-20005) | INSERT trực tiếp 25.000.000đ vượt hạn mức VI005 bị Compound Trigger chặn với mã -20005, không phát sinh ORA-04091. | Đạt |




***Hình 4.1. Kết quả đối soát Explicit Cursor trên 10 ví (0 ví lệch) qua giao diện Web***

### 4.3. Phân tích và tối ưu hoá Truy vấn 1 (Thống kê giao dịch 30 ngày)

Trong tệp sql/10_optimization_plans.sql, nhóm chọn Truy vấn 1 là tác vụ tra cứu số lượng giao dịch và tổng chi tiêu thành công của ví VI001 trong 30 ngày gần nhất — truy vấn được gọi liên tục trên màn hình Dashboard và hàm kiểm tra hạn mức.

**Đoạn mã 4.1. So sánh câu lệnh Truy vấn 1 Trước và Sau tối ưu hoá**

```
-- TRUOC TOI UU: Khong co Composite Index, ham TRUNC(NGAY_TAO) vo hieu hoa Index
(Non-Sargable)
SELECT COUNT(*) AS SO_GIAO_DICH, NVL(SUM(SO_TIEN), 0) AS TONG_CHI_TIEU
  FROM GIAO_DICH
 WHERE MA_VI_NGUON = 'VI001'
   AND TRANG_THAI  = 'THANH_CONG'
   AND TRUNC(NGAY_TAO) >= TRUNC(SYSDATE - 30);

-- SAU TOI UU: Composite Covering Index IDX_GD_VI_NGUON_NGAY_TT(MA_VI_NGUON,
TRANG_THAI, NGAY_TAO DESC, SO_TIEN)
-- Ket hop viet lai dieu kien dai thoi gian Sargable
SELECT COUNT(*) AS SO_GIAO_DICH, NVL(SUM(SO_TIEN), 0) AS TONG_CHI_TIEU
  FROM GIAO_DICH
 WHERE MA_VI_NGUON = 'VI001'
   AND TRANG_THAI  = 'THANH_CONG'
   AND NGAY_TAO   >= CAST(TRUNC(SYSDATE - 30) AS TIMESTAMP);
```

Phân tích Execution Plan cho thấy hai nguyên nhân làm tăng chi phí ở phiên bản trước tối ưu: thứ nhất, việc bọc hàm TRUNC(NGAY_TAO) lên cột bảng khiến Oracle CBO không thể sử dụng B-Tree Index thông thường trên NGAY_TAO; thứ hai, bảng GIAO_DICH chưa có chỉ mục bao phủ (Covering Index) nên CBO buộc phải thực hiện TABLE ACCESS FULL quét toàn bộ hơn 15.000 dòng dữ liệu. Sau khi tạo chỉ mục IDX_GD_VI_NGUON_NGAY_TT chứa đủ 4 cột (MA_VI_NGUON, TRANG_THAI, NGAY_TAO DESC, SO_TIEN) và viết lại điều kiện so sánh trực tiếp trên cột NGAY_TAO, Oracle CBO chuyển sang kế hoạch INDEX RANGE SCAN và tính tổng trực tiếp trên các leaf block của chỉ mục mà hoàn toàn không cần đọc vào bảng gốc (0 Table Access by Index Rowid).

### 4.4. Phân tích và tối ưu hoá Truy vấn 2 (Đối soát Nợ/Có sổ cái 90 ngày)

Truy vấn 2 là báo cáo đối soát tổng phát sinh GHI_CO và GHI_NO của từng ví kết hợp họ tên nhân viên trong 90 ngày gần nhất trên bảng CHI_TIET_SO_CAI (hơn 30.000 dòng). Cách viết ban đầu sử dụng 2 Correlated Subquery độc lập ngay trong

danh sách SELECT kèm hàm TRUNC(sc.NGAY_GHI_SO), khiến bảng sổ cái có nguy cơ bị quét lặp lại nhiều lần theo từng ví.

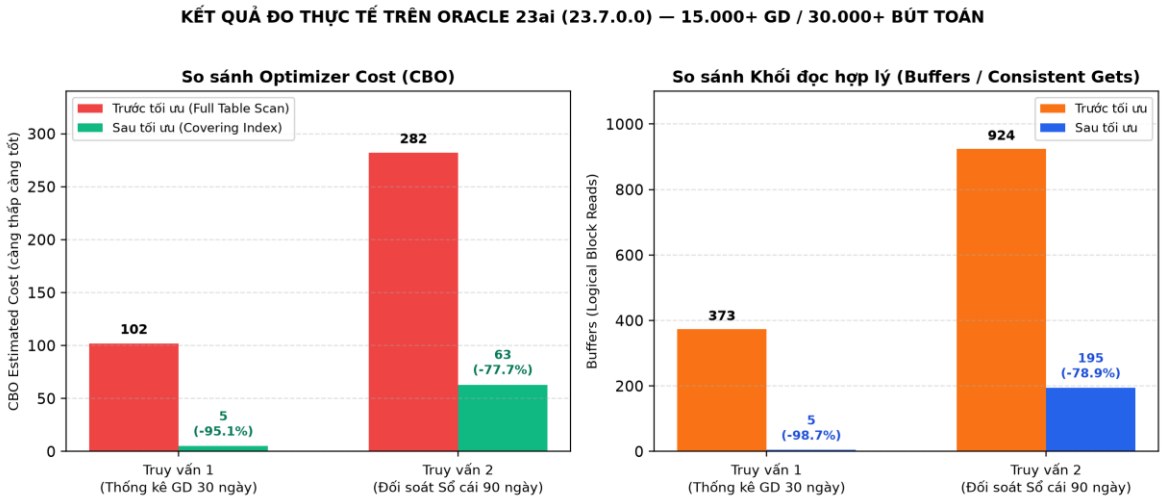
**Đoạn mã 4.2. So sánh Truy vấn 2: Correlated Subquery (Trước) vs Conditional Aggregation + Index (Sau)**

```
-- TRUOC TOI UU: 2 Correlated Subquery lap lai + TRUNC(NGAY_GHI_SO)
SELECT v.MA_VI, nv.HO_TEN,
       (SELECT NVL(SUM(sc.SO_TIEN), 0) FROM CHI_TIET_SO_CAI sc
         WHERE sc.MA_VI = v.MA_VI AND sc.LOAI_BUT_TOAN = 'GHI_CO'
           AND TRUNC(sc.NGAY_GHI_SO) >= TRUNC(SYSDATE - 90)) AS TONG_GHI_CO,
       (SELECT NVL(SUM(sc.SO_TIEN), 0) FROM CHI_TIET_SO_CAI sc
         WHERE sc.MA_VI = v.MA_VI AND sc.LOAI_BUT_TOAN = 'GHI_NO'
           AND TRUNC(sc.NGAY_GHI_SO) >= TRUNC(SYSDATE - 90)) AS TONG_GHI_NO
  FROM VI_DIEN_TU v JOIN NHAN_VIEN nv ON v.MA_NV = nv.MA_NV;

-- SAU TOI UU: Gom nhom 1 luot (Single-Pass Conditional Aggregation) + Covering
Index IDX_SC_VI_NGAY_LOAI
SELECT v.MA_VI, nv.HO_TEN,
       NVL(agg.TONG_GHI_CO, 0) AS TONG_GHI_CO,
       NVL(agg.TONG_GHI_NO, 0) AS TONG_GHI_NO
  FROM VI_DIEN_TU v
  JOIN NHAN_VIEN nv ON v.MA_NV = nv.MA_NV
  LEFT JOIN (
      SELECT sc.MA_VI,
             SUM(CASE WHEN sc.LOAI_BUT_TOAN = 'GHI_CO' THEN sc.SO_TIEN ELSE 0 END)
AS TONG_GHI_CO,
             SUM(CASE WHEN sc.LOAI_BUT_TOAN = 'GHI_NO' THEN sc.SO_TIEN ELSE 0 END)
AS TONG_GHI_NO
        FROM CHI_TIET_SO_CAI sc
       WHERE sc.NGAY_GHI_SO >= CAST(TRUNC(SYSDATE - 90) AS TIMESTAMP)
       GROUP BY sc.MA_VI
  ) agg ON v.MA_VI = agg.MA_VI;
```

*Bảng 4.2. Kết quả đo Execution Plan thực tế trên Oracle Database 23ai (sql/10_optimization_plans.sql)*

| Truy vấn | Kế hoạch & Chi phí Trước tối ưu | Kế hoạch & Chi phí Sau tối ưu | Nguyên lý giảm chi phí CBO |
|---|---|---|---|
| Truy vấn 1 (Thống kê 30 ngày) | TABLE ACCESS FULL (GIAO_DICH)<br>Cost = 102 \| Buffers = 373 | INDEX RANGE SCAN (Covering) IDX_GD_VI_NGUON_NGAY_TT<br>Cost = 5 (-95,1%)<br>Buffers = 5 (-98,7%) | Bỏ TRUNC(NGAY_TAO) và đưa SO_TIEN vào cuối chỉ mục giúp đọc trọn vẹn trong 5 block lá. |
| Truy vấn 2 (Đối soát 90 ngày) | 2x VW_SSQ + TABLE ACCESS FULL (CHI_TIET_SO_CAI)<br>Cost = 282 \| Buffers = 924 | 1x INDEX FAST FULL SCAN IDX_SC_VI_NGAY_LOAI + HASH GROUP<br>Cost = 63 (-77,7%)<br>Buffers = 195 (-78,9%) | Gom nhóm 1 lượt (Conditional Aggregation) kết hợp Covering Index thay thế 2 lần quét toàn bảng. |




*Hình 4.2. Biểu đồ đo thực tế mức giảm Optimizer Cost và Buffers trên Oracle Database 23ai*

**Đoạn mã 4.3. Trích xuất kết quả DBMS_XPLAN thực tế từ container Oracle Database 23ai (23.7.0.0)**

```
-- TRUY VAN 1 (ALLSTATS LAST): Truoc (Buffers=373, Cost=102) -> Sau (Buffers=5, Cost=5)
| Id | Operation         | Name                  | Starts | E-Rows | A-Rows | Buffers |
|  0 | SELECT STATEMENT  |                       |      1 |        |      1 |       5 |
|  1 |  SORT AGGREGATE   |                       |      1 |      1 |      1 |       5 |
|* 2 |   INDEX RANGE SCAN| IDX_GD_VI_NGUON_NGAY_TT |      1 |    493 |    492 |       5 |

-- TRUY VAN 2 (ALLSTATS LAST): Truoc (Buffers=924, Cost=282) -> Sau (Buffers=195, Cost=63)
| Id | Operation           | Name                | Starts | E-Rows | A-Rows | Buffers |
|  0 | SELECT STATEMENT    |                     |      1 |        |     10 |     195 |
|  2 |  HASH JOIN OUTER    |                     |      1 |     10 |     10 |     195 |
| 12 |   HASH GROUP BY     |                     |      1 |     10 |      8 |     187 |
|*13 |    INDEX FAST FULL SC| IDX_SC_VI_NGAY_LOAI |      1 |  14707 |  15039 |     187 |
```

Kết quả thực nghiệm trên container Oracle Database 23ai Free (phiên bản 23.7.0.0) khẳng định độ chính xác cao của mô hình ước lượng Oracle CBO sau khi chạy DBMS_STATS: tại Truy vấn 1, CBO ước lượng E-Rows = 493 dòng so với thực tế A-Rows = 492 dòng, chi phí giảm từ 102 xuống 5 (-95,1%) và số khối đọc giảm từ 373 xuống 5 (-98,7%). Tại Truy vấn 2, chi phí giảm từ 282 xuống 63 (-77,7%) và tổng khối đọc hợp lý giảm từ 924 xuống 195 (-78,9%).

# CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 5.1. Kết quả đạt được

Đối chiếu với các tiêu chí kỹ thuật bắt buộc và Rubric đánh giá của học phần, Nhóm 12 đã hoàn thành đầy đủ các hạng mục trọng tâm của Đề tài 08:

- **Về hạ tầng và thiết kế CSDL:** Đóng gói hoàn chỉnh Docker Oracle Database 23ai Free với cấu hình tiếng Việt AL32UTF8; chuẩn bị 2 bộ script đồng bộ cho Docker SQL*Plus (sql/) và DBeaver Alt+X (sql_dbeaver/); thiết kế lược đồ 9 bảng chuẩn 3NF kèm Sổ cái kế toán kép.
- **Về kỹ thuật cốt lõi PL/SQL & Exception Handling:** Cài đặt hoàn chỉnh Package PKG_NGOAI_LE_VI (10 mã lỗi tuỳ biến -20001..-20010, ghi log độc lập bằng PRAGMA AUTONOMOUS_TRANSACTION), 5 Functions (gồm hàm BOOLEAN chuẩn SQL của Oracle 23ai), Package PKG_QUAN_LY_VI (7 thủ tục nghiệp vụ khoá chống Deadlock, Idempotency, Explicit Cursor) và 3 Triggers (gồm Compound Trigger).
- **Về ứng dụng minh hoạ 3 tầng và tối ưu hoá:** Xây dựng hoàn chỉnh REST API Node.js + Express + oracledb Thin Mode và Giao diện Web cho phép thao tác toàn bộ luồng nghiệp vụ, mô phỏng nhanh 6 tình huống ngoại lệ, kiểm thử Race Condition song song và phân tích Execution Plan.

## 5.2. Hạn chế và hướng phát triển

Bên cạnh các kết quả đạt được, hệ thống hiện vẫn còn một số điểm giới hạn phù hợp với phạm vi môn học: (1) Giao diện hiện tại tập trung minh hoạ luồng dữ liệu và kiểm thử ngoại lệ nên chưa tích hợp xác thực đa yếu tố (OTP/PIN giao dịch) và phân quyền chi tiết theo vai trò; (2) Chưa kết nối hàng đợi bất đồng bộ (như Kafka/RabbitMQ) cho kịch bản hàng nghìn giao dịch bùng nổ tức thời.

Trong giai đoạn phát triển tiếp theo, nhóm định hướng mở rộng: tích hợp chữ ký số/OTP xác nhận giao dịch giá trị lớn; khai thác thêm tính năng Oracle 23ai MLE (Multilingual Engine – JavaScript) để viết bộ quy tắc chấm điểm rủi ro gian lận linh hoạt ngay trong CSDL; và bổ sung bảng phân vùng (Range Partitioning) theo tháng trên bảng GIAO_DICH khi quy mô dữ liệu vượt ngưỡng hàng triệu dòng.

## 5.3. Phân công và đóng góp của từng thành viên Nhóm 12

Theo quy định tại Mục 4 của hướng dẫn đồ án, bảng dưới đây ghi nhận chi tiết phần việc phụ trách và tỷ lệ đóng góp thực tế của từng thành viên Nhóm 12 làm cơ sở đối chiếu trong phần vấn đáp cá nhân:

*Bảng 5.1. Tổng hợp đóng góp chi tiết của các thành viên Nhóm 12*

| STT | Thành viên / MSSV | Nội dung công việc phụ trách chi tiết | Đóng góp |
|---|---|---|---|
| 1 | Nguyễn Quốc Huy (24DH113268) Nhóm trưởng | • Thiết kế kiến trúc tổng thể, Docker Compose Oracle 23ai & DBeaver.<br>• Thiết kế chuẩn hoá 3NF (9 bảng) và ràng buộc toàn vẹn.<br>• Xây dựng PKG_NGOAI_LE_VI (PRAGMA EXCEPTION_INIT, AUTONOMOUS_TRANSACTION) và SP_CHUYEN_TIEN / SP_HOAN_TIEN.<br>• Tổng hợp kiểm thử Race Condition, tối ưu Execution Plan & rà soát báo cáo. | 34% |
| 2 | Phạm Thành Đức (24DH113249) | • Cài đặt 5 Stored Functions (tính phí, hạn mức ngày, SQL BOOLEAN Oracle 23ai).<br>• Cài đặt SP_NAP_TIEN, SP_RUT_TIEN, SP_THANH_TOAN_DICH_VU và Explicit Cursor đối soát SP_DOI_SOAT_HANG_LOAT.<br>• Cài đặt 3 Triggers (gồm Compound Trigger chống Mutating Table ORA-04091).<br>• Viết kịch bản dữ liệu mẫu 15.000+ giao dịch và Chương 2, Chương 4 báo cáo. | 33% |
| 3 | Nguyễn Thế Vinh (24DH141906) | • Xây dựng lớp Backend API (Node.js + Express + oracledb Thin Mode) và các endpoint OUT Binds.<br>• Thiết kế và lập trình Giao diện Web minh hoạ (Dashboard, Ví, Giao dịch, Exception Lab, Đối soát).<br>• Thực hiện bộ 9 kịch bản kiểm thử sql/09_test_scenarios.sql và vẽ các sơ đồ kiến trúc, ERD, lưu đồ.<br>• Biên tập Chương 3, Chương 5 và chuẩn hoá định dạng tài liệu Word. | 33% |

## 5.4. Định hướng chuẩn bị vấn đáp theo Ngân hàng câu hỏi học phần

Nhằm sẵn sàng cho 3 vòng vấn đáp (Giải thích mã nguồn – Tình huống mở rộng – Thao tác trực tiếp trên máy), nhóm đã hệ thống hoá các câu hỏi trọng tâm dành cho Nhóm PL/SQL và Tối ưu hoá:

- **Vì sao chọn Explicit Cursor trong SP_DOI_SOAT_HANG_LOAT thay vì Cursor FOR Loop?** Explicit Cursor giúp kiểm soát tường minh trạng thái mở/đóng con trỏ (%ISOPEN, %NOTFOUND), xử lý an toàn trường hợp phát sinh lỗi giữa vòng lặp để đóng con trỏ tránh rò rỉ bộ nhớ, và dễ mở rộng sang FETCH ... BULK COLLECT LIMIT khi dữ liệu lớn.
- **Nếu một ngoại lệ không được bắt trong thủ tục thì điều gì xảy ra với giao dịch đang chạy?** Oracle sẽ tự động ROLLBACK toàn bộ câu lệnh DML nằm trong lời gọi thủ tục đó và ném lỗi thô (unhandled exception) ra tầng API. Việc bắt ngoại lệ tường minh giúp ta sử dụng SAVEPOINT để khôi phục chính xác, ghi log độc lập bằng AUTONOMOUS_TRANSACTION và trả mã lỗi chuẩn hoá (-20001..-20010) về giao diện.
- **Điều gì xảy ra nếu bỏ Compound Trigger và thay bằng Row-level Trigger truy vấn GIAO_DICH?** Ngay khi chèn hoặc cập nhật trên bảng GIAO_DICH, Oracle sẽ ném lỗi ORA-04091 (table is mutating) do không cho phép đọc lại chính bảng đang bị biến đổi ở mức từng dòng. Compound Trigger khắc phục bằng cách thu thập mã ví ở pha AFTER EACH ROW và chỉ truy vấn tổng hợp ở pha AFTER STATEMENT.

## TÀI LIỆU THAM KHẢO

[1] Oracle Corporation, "Oracle Database 23ai PL/SQL Language Reference," Oracle Help Center, 2024. (Tài liệu đặc tả cú pháp gói PL/SQL, PRAGMA EXCEPTION_INIT, AUTONOMOUS_TRANSACTION và Compound Triggers).

[2] Oracle Corporation, "Oracle Database 23ai SQL Language Reference – BOOLEAN Data Type and Indexing," Oracle Help Center, 2024. (Tham khảo kiểu dữ liệu BOOLEAN gốc trong SQL trên phiên bản 23ai và thiết kế chỉ mục tổng hợp).

[3] Oracle Corporation, "node-oracledb Documentation (Thin Mode Architecture & PL/SQL Bind Parameters)," GitHub / Oracle, 2025. (Hướng dẫn cấu hình Connection Pool và truyền tham số OUT Binds giữa Node.js và PL/SQL).

[4] Lê Thị Minh Nguyện, "Bài giảng và Hướng dẫn thực hành Hệ quản trị Cơ sở dữ liệu (Oracle Database 23ai)," Khoa Công nghệ Thông tin, Trường Đại học Ngoại ngữ – Tin học TP.HCM (HUFLIT), 2026.

[5] S. Feuerstein and B. Pribyl, Oracle PL/SQL Programming, 6th ed. Sebastopol, CA, USA: O'Reilly Media, 2014. (Tham khảo mẫu thiết kế xử lý ngoại lệ tập trung và tránh lỗi Mutating Table trong hệ thống OLTP).

[6] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016. (Cơ sở lý thuyết chuẩn hoá lược đồ quan hệ đến dạng chuẩn 3NF và kiểm soát đồng thời ACID).

# PHỤ LỤC: HƯỚNG DẪN TRIỂN KHAI DOCKER, DBEAVER VÀ KỊCH BẢN DEMO

## Phụ lục A. Trình tự triển khai hệ thống trên Docker và DBeaver

Toàn bộ mã nguồn dự án được tổ chức thành 5 thành phần liên kết chặt chẽ gồm tệp docker-compose.yml, thư mục sql/ (11 tệp dùng dấu / cho SQL*Plus và Docker), thư mục sql_dbeaver/ (11 tệp đồng bộ đã bỏ dấu / đứng riêng cho DBeaver Alt+X), thư mục backend/ (Node.js API và Giao diện Web) cùng tập lệnh scripts/run_sql_docker.ps1. Mã nguồn đầy đủ được lưu trữ công khai tại kho lưu trữ GitHub nhóm:

https://github.com/TheDay0811/huflit-oracle23ai-ewallet-nhom12

- **Bước 1 – Khởi động Oracle Database 23ai trên Docker:** Mở Docker Desktop, khởi động container oracle23ai có sẵn (docker start oracle23ai) hoặc chạy docker compose up -d tại thư mục gốc dự án để tạo container oracle23ai_ewallet. Tiếp đó chạy .\scripts\run_sql_docker.ps1; tập lệnh tự động phát hiện container đang chạy và nạp đủ 9 bảng, 15.000+ giao dịch mẫu, Packages, Functions, Triggers cùng kết quả 9 kịch bản kiểm thử và Execution Plan.
- **Bước 2 – Kết nối và chạy kiểm tra trên DBeaver:** Tạo kết nối Oracle mới trên DBeaver trỏ tới localhost:1521 với Service Name là FREEPDB1 và đăng nhập trực tiếp bằng tài khoản sở hữu schema EWALLET_APP / Ewallet2026 (tránh dùng user khác gây trống danh sách Tables). Bật cửa sổ Server Output (Ctrl+Shift+O), mở sql_dbeaver/09_test_scenarios.sql và nhấn Alt+X để chạy mượt mà 9 kịch bản kiểm thử.
- **Bước 3 – Chạy ứng dụng Web minh hoạ:** Mở terminal tại thư mục backend/, chạy npm install và npm start. Truy cập http://localhost:3000 trên trình duyệt để kiểm tra trạng thái kết nối xanh lá Oracle 23ai (FREEPDB1).

### Phụ lục mã A.1. Các lệnh PowerShell triển khai và chạy lại toàn bộ script Oracle 23ai

```
# 1. Khoi dong container Oracle 23ai co san hoac tao moi bang Docker Compose
docker start oracle23ai   # Hoac: docker compose up -d

# 2. Chay toan bo 11 script SQL + 9 Kich ban kiem thu + Execution Plan
.\scripts\run_sql_docker.ps1

# 3. Khoi dong API & Giao dien Web (http://localhost:3000)
cd backend
npm install
npm start
```

## Phụ lục B. Kịch bản Demo trực tiếp khi bảo vệ đồ án (8–10 phút)

*Bảng A.1. Kịch bản trình bày và demo trực tiếp hệ thống trong buổi bảo vệ*

| Thời lượng | Nội dung trình diễn | Điểm nhấn kỹ thuật cần làm rõ |
|---|---|---|
| Phút 0–2 | Tổng quan & Kiến trúc | Trình bày bài toán ví nội bộ, chỉ rõ 3 tầng Web ↔ Node.js oracledb ↔ Docker Oracle 23ai và trạng thái kết nối PDB FREEPDB1. |
| Phút 2–4 | Demo DBeaver & PL/SQL lõi | Mở DBeaver (kết nối EWALLET_APP) hiển thị 9 bảng 3NF, Package PKG_NGOAI_LE_VI (-20001..-20010), chạy truy vấn trong sql_dbeaver/ gọi FN_TINH_PHI_GIAO_DICH và FN_KIEM_TRA_VI_HOAT_DONG. |
| Phút 4–6 | Demo Giao dịch & Exception Lab | Chuyển tiền hợp lệ VI001 → VI002; bấm 'Gửi lại mã tham chiếu' kích hoạt -20002; vào Exception Lab thử -20001 (VI007), -20003 (VI006), Race Condition song song và chỉ bảng NHAT_KY_LOI_GIAO_DICH. |
| Phút 6–8 | Demo Trigger, Hoàn tiền & Đối soát | Khoá/Mở ví trên Web để hiện Audit Log do Trigger ghi; hoàn tiền 1 giao dịch; chạy Đối soát Explicit Cursor (0 ví lệch) và mở sql/10_optimization_plans.sql trình bày Execution Plan. |